GRADUATION THESIS

WAREHOUSE MANAGEMENT SYSTEM DESIGN FOR SMART WAREHOUSE

NGUYỄN PHÚ QUYỀN
quyen.np192246@sis.hust.edu.vn

Advanced Program of Control and Automation Engineering

Instructor: M.Sc. Đinh Thị Lan Anh

Instructor's signature

Department: Automation
School: Electrical and Electronic Engineering

Hanoi, 03/2023

ACKNOWLEDGEMENT

My heartfelt gratitude goes to Ms. Dinh Thi Lan Anh for her invaluable guidance and assistance. Her support was crucial in helping me overcome various obstacles throughout the process. Without her help and the favorable conditions she provided, I wouldn't have been able to finish this project.

I would like to express my deep gratitude to all the teachers at the School of Electrical Engineering, and those from other schools, for imparting their knowledge to me over the past four years. The lessons I have learned during my time in university will serve as my guide, pointing me in the right direction for my future endeavors.

Although I have put forth my best, my abilities and knowledge regarding this matter are limited. Hence, errors are inevitable. Therefore, I would greatly appreciate your guidance and any additional suggestions you may have to improve this project.

Hanoi, date 28, month 03, year 2023

Student

Nguyễn Phú Quyền

ABSTRACT

Vietnam's warehouse management systems have undergone significant changes over the years, driven by a combination of factors such as economic growth, globalization, and technological advancements. AS Vietnam rapidly transforming into a manufacturing and logistics hub for the region, there is a growing need for modern and efficient warehouses. As a result, warehouse management strategies have become a popular topic among economists, scientists, and engineers.

Tien Phong Plastic Joint Stock Company is a top plastic manufacturer in Vietnam that produces various plastic products for different industries. To efficiently manage their warehouse operations, having a warehouse management system is crucial for their factories and warehouses.

The objective of this thesis is to initially examine the structure and process of Tien Phong's warehouse. Following that, it involves creating models and assessing the significance of warehouse operations to facilitate optimization and management. Lastly, it proposes remedies and devises an independent warehouse management system for Tien Phong Plastic Joint Stock Company's warehouse.

CONTENTS

LIST OF ACRONYMS

WMS	Warehouse Management System
TPP	Tien Phong Plastic Joint Stock Company
RFID	Radio Frequency Identification
SQL	Structured Query Language
ABC	ABC Analysis
SKU	Stock Keeping Unit

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1. INTRODUCTION

1.1 Smart warehouse

A warehouse is a large building primarily used for storing and distributing goods. It typically includes shelves or racks for storing products until they are needed for shipping or other purposes. [1]

A smart warehouse, on the other hand, uses advanced technologies such as RFID tags, autonomous robots or drones for material handling, and predictive analytics to forecast demand and optimize supply chain operations. Data analytics may also be used to monitor and improve warehouse processes while reducing waste. [2]

The ultimate aim of a smart warehouse is to create a highly efficient and responsive supply chain that can quickly adapt to changing market demands and customer needs.

The main objectives of warehouse management activities can be listed as:

- Quality management.
- Quantity management.
- Space optimization.
- Resource optimization.
- Synchronization.

In Vietnam, handling plastic production in companies can be difficult due to the limited technology and various categories involved. Most warehouse management systems in the country depend on manual labor, which can be inefficiency, costly, and time-consuming. Plastic products come in different sizes, shapes, and quantities, leading to high storage costs. Additionally, customer requirements for plastic products are unpredictable and diverse, resulting in unstable warehouse management. Balancing customer satisfaction with storage expenses can be a daunting task. [3]

To overcome these challenges, it is imperative to adopt an appropriate Warehouse Management System (WMS). With a WMS in place, inventory accuracy is improved and labor costs are reduced.

1.2 Project objective

The primary goal of the project is to suggest appropriate solutions for the management of the warehouse system. In order to accomplish this objective, a number of tasks must be carried out:

- Analyzing and evaluating the current layout and operations.
- Identifying the specifications of plastic products that should be considered.

– Deriving a suitable system model and quantifying values based on the current warehouse.
– Suggesting improvements to enhance efficiency while considering existing processes.
– Developing a warehouse management system that meets the identified specifications and requirements.

1.3 Thesis approach

The approach to the topic is rooted in the scientific process, which ensures objectivity and reliability. The process begins with an overview of research related to warehouse management to identify new trends. The current performance efficiency of the chosen warehouse is then evaluated to determine its status. Based on this data, an appropriate management method for the warehouse is selected and applied. After the development process, opportunities and benefits are considered for deploying the new system application to the current warehouse. Finally, the new warehouse management system is developed and tested.

During the development of the new system, various techniques and methods were employed. Theoretical research was conducted by reviewing domestic and international literature related to smart warehouse management systems. This helped identify key factors for the management system and successful experiences of typical enterprises and corporations. Theoretical research is an effective and scientific method to discover information, identify research gaps, and provide a feasible implementation strategy for actual conditions.

After conducting theoretical research, appropriate designs for equipment and software were proposed based on the major requirements and analysis of the warehouse's operations. As the complex requirements of a plastic warehouse cannot be met by existing management software options, it was decided that suitable software would be developed according to the characteristics of the plastic industry. Two options were considered: web-based or traditional software on a computer. Both options have open sources and frameworks that support development, but a web-based application was chosen over software due to its ability to run directly on many devices without requiring installation on every computer.

The last technique employed involves analyzing data gathered from associations, organizations, and units to assess the level of conformity of enterprises with the requirements of the WMS. The gathered information is analyzed based on parameters such as the number and size of the enterprise, its business field, management capacity, infrastructure, technology level, as well as production and business status to determine their suitability for the WMS.

CHAPTER 2. OVERVIEW

2.1 Tien Phong warehouse

2.1.1 Tien Phong Plastic Joint Stock Company

Tien Phong Plastic Joint Stock Company is a prominent manufacturer and distributor of plastic products in Vietnam [4]. Founded in 1975, it has since become one of Vietnam's biggest plastic manufacturers, offering a diverse range of products, including plastic pipes, fittings, and packaging materials.

2.1.2 Tien Phong warehouse

2.1.2.1 Layout

Based on the information in the appendix (see Figure A.1 and Figure A.2), it appears that there is currently no shelf or rack system in the warehouse. This can lead to difficulties in locating and retrieving items promptly, resulting in longer order fulfillment times and negatively impacting overall warehouse operations. While not mandatory, implementing a shelf or rack system can significantly improve the efficiency and effectiveness of the warehouse. [5]

In order to develop a suitable system and design solution in the later chapters, a custom rack system is recommended. An orthographic projection of each location with a rack system suitable for the carton box (size 40x60x40) has been designed.

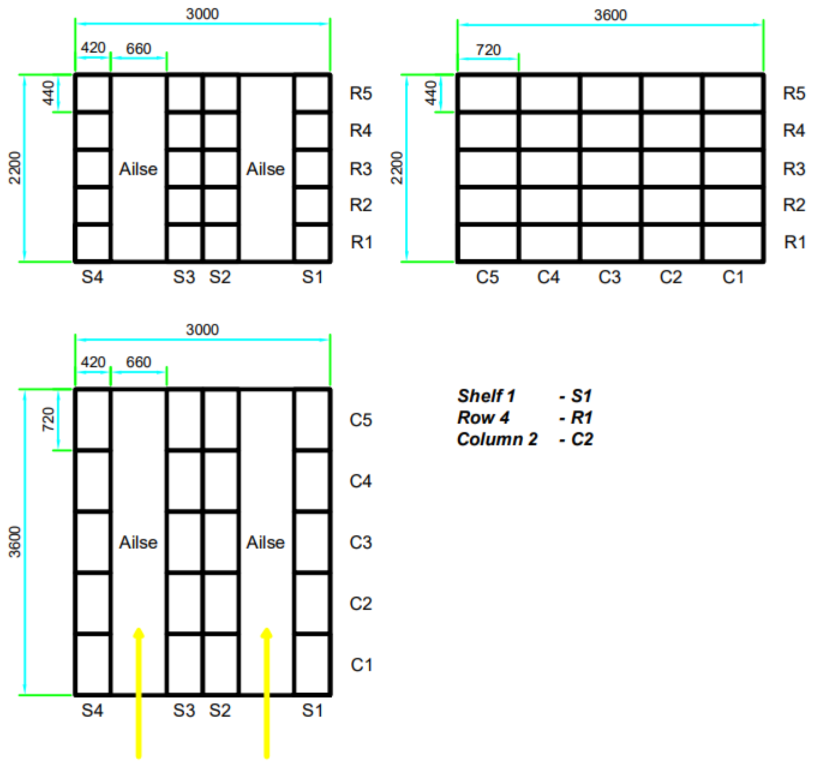


Figure 2.1. Orthographic projection of one location

In reference to Figure 2.1., four shelves with 25 slots each (5x5) have been incorporated. Furthermore, there are two aisles (0.66 meters) between the shelves at each location.

After recalculations, the storage capacity of each unit is as follows:

- Slot: 0.13 cubic meters;
- Shelf: 3.33 cubic meters;
- Location: 13.31 cubic meters;
- All: 2395.01 cubic meters.

It is crucial to take into account orthographic projections of sections in addition to the overall layout. By analyzing the sections of the warehouse, it is possible to gain a more detailed understanding of how the space can be utilized most effectively.

Figure 2.2. Top projection of sections

TPP's warehouse includes a total of:

- 3 floors;
- 8 sections;
- 180 locations;

- 720 shelves;
- 18000 slots.

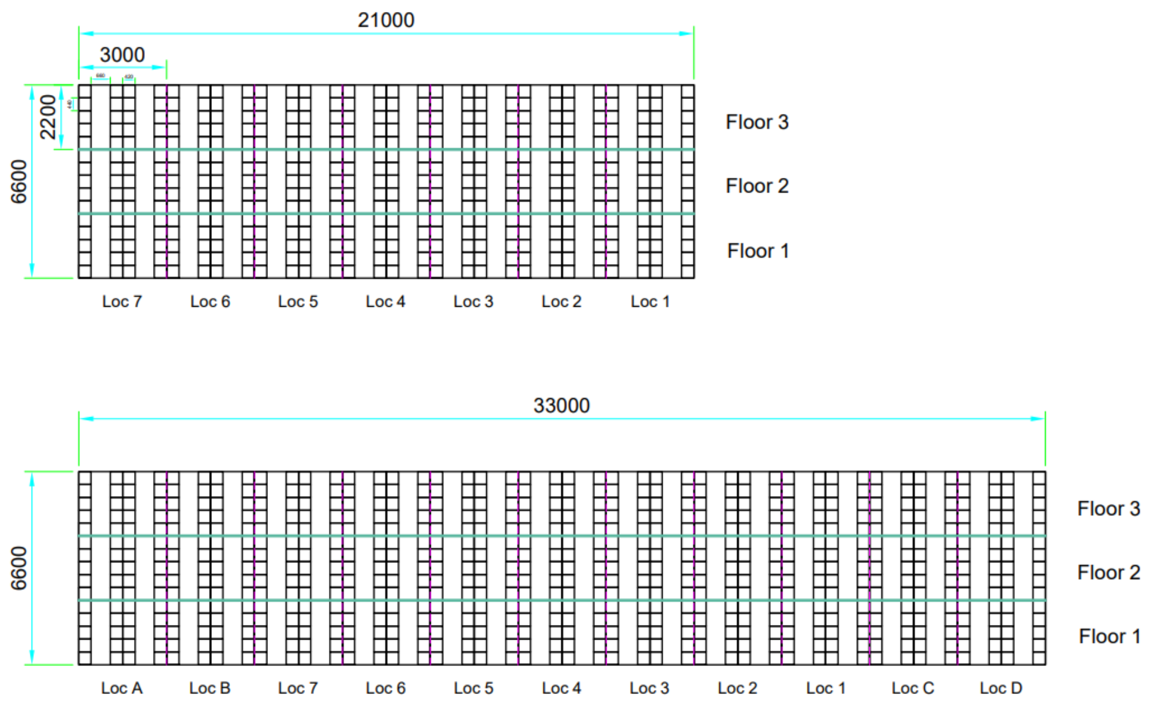


Figure 2.3. Front projection of sections

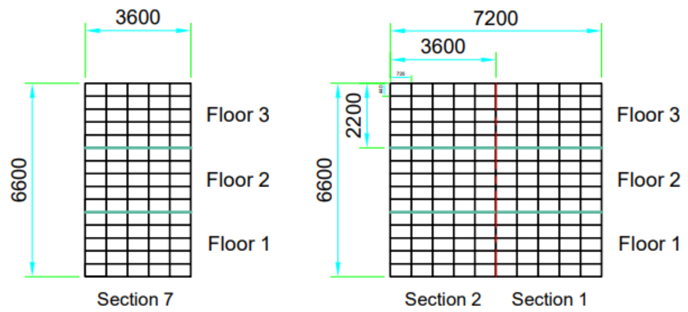


Figure 2.4. Side projections of sections

2.1.2.2 Process

According to the TPP warehouse report (Figure A.4 and A.5), it appears that the current management processes mainly rely on manual labor, which is both time-consuming and costly. In the span of 10 months, a total of 1,008,832 products (Figure A.7) were found to be sitting idly in storage in 4 out of the 8 sections. Additionally, the price of each product varies significantly, from a few thousand to several million Vietnamese dong. The cost of handling such a considerable backlog of products is significant, highlighting the necessity for a more efficient and streamlined management process.

Based on the analysis of statistics and research data, it is estimated that ten changes per year will occur on average for each slot with the implementation of the new rack system. This estimate takes into account various factors such as the size and type of items being stored and the expected frequency of inventory turnover. [6]

2.2 Warehouse management system

The process of organizing, controlling, and managing a warehouse's operations is known as warehouse management. It involves handling the receipt, storage, and distribution of goods and materials. The goal of warehouse management is to make the most of space, labor, and equipment to ensure that products are stored, tracked, and delivered efficiently. [7]

Software applications called warehouse management systems (WMS) are used to manage warehouse operations. These systems provide real-time information about inventory levels, order processing, and shipping. WMS utilizes technologies such as barcode scanners, RFID tags, and automated material handling equipment to enhance warehouse operations. [7]

The use of a WMS can provide several benefits, including:

- Improved inventory accuracy: A WMS provides real-time information about inventory levels, reducing the risk of overstocking or understocking.
- Increased efficiency: By automating warehouse operations, a WMS can reduce the time and labor required to manage inventory, orders, and shipping.
- Enhanced customer service: A WMS can help ensure that products are delivered to customers on time and in good condition, improving customer satisfaction.
- Cost savings: By optimizing warehouse operations, a WMS can reduce labor costs, minimize inventory holding costs, and reduce shipping expenses.

2.3 Management Process

2.3.1 Receiving and put-away

Although warehouses differ in terms of size, type, function, ownership, and location some fundamental processes remain (Figure 2.5.).

2.3.1.1 Receiving

Receiving, goods-in or in-handling is a crucial process within the warehouse. Ensuring that the correct product has been received in the right quantity and in the right condition at the right time is one of the mainstays of the warehouse operation. These elements are often termed supplier compliance.

2.3.1.2 In-handling

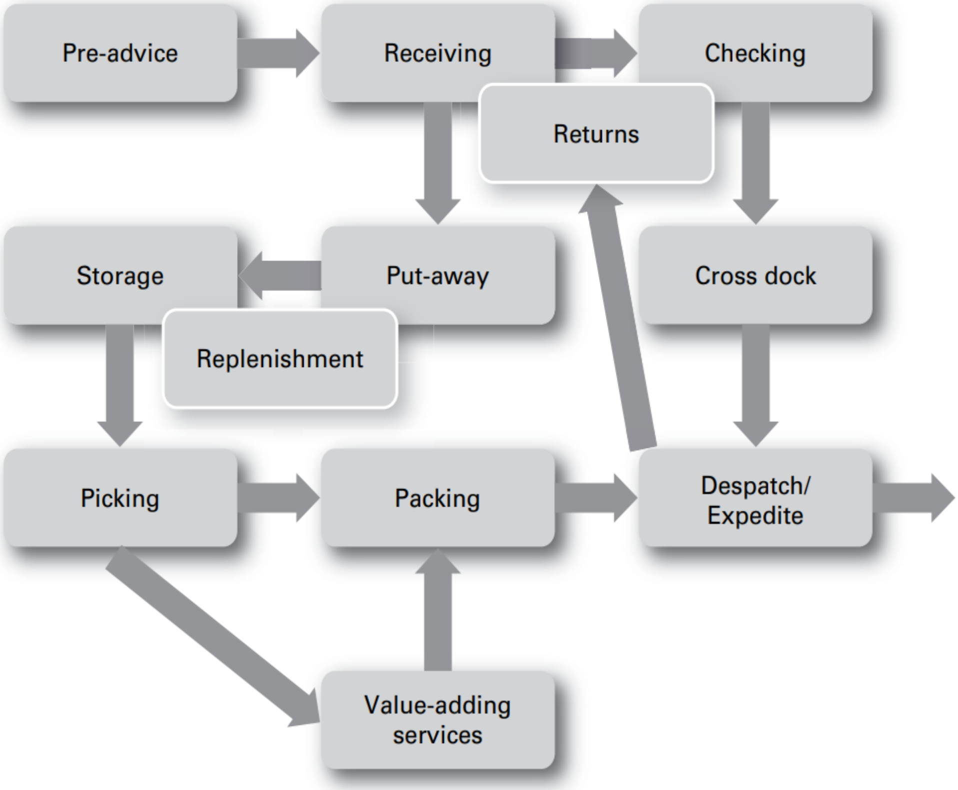


Figure 2.5. Receiving and put-away processes

One Of the main challenges for a warehouse manager is to match labour hours with work content. Handling a product the least amount of time possible (labour touch points) leads to reduced labour hours and as a consequence, reduced cost. [8]

Depending on the operation, labour can be the single biggest cost within a warehouse. It can be between 48 and 60 percents Of the total warehouse cost depending on the amount Of automation utilized. It is also the most difficult cost to control. [8]

In-handling makes up approximately 20 per cent of the total direct labour cost within a retail warehouse. [8]

2.3.1.3 Put-away

Many of today's WMSs allocate product locations in advance and instruct the operator as to where to place the goods. This can be directly to the despatch area if the product is to be cross docked as discussed above, to the pick face as a form of replenishment or to a reserve or bulk-storage location.

In order for this system to work effectively, a great deal of information needs to be programmed into the system. This includes the following:

- size, weight and height of palletized goods;
- results of an ABC analysis
- family product groups;

- actual sales combinations;
- current status of pick face for each product.

2.3.2 Order picking

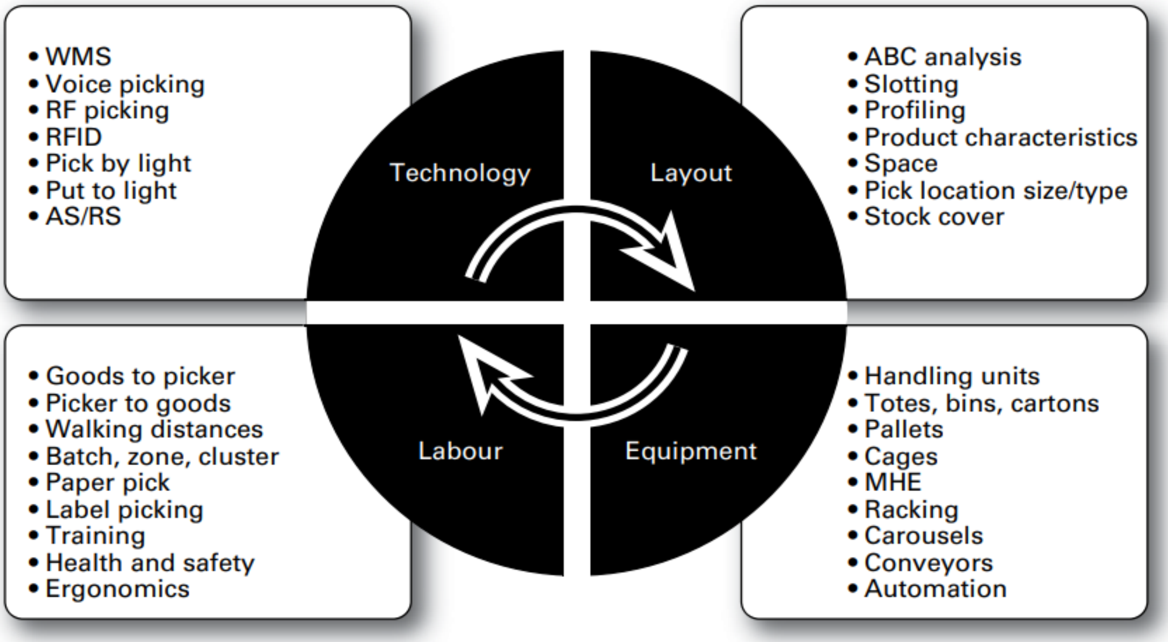


Figure 2.6. Picking relationships

Order picking is the most costly activity within today's warehouses. Not only is it labour intensive, but it is challenging to automate, can be difficult to plan, is prone to error and crucially has a direct impact on customer service (see Figure 2.6.). Typical errors include omitting items from the order, sending the wrong item and sending the wrong number of items.

2.4 Storage strategies

The organization of warehouses, inputs, and outputs is influenced by what is known as "storage strategies". While these strategies are not primarily focused on what and where items are stored, they do prioritize the structural arrangement of these elements. The most common ones are: FIFO, LIFO, FEFO, HIFO, ...

2.4.1 FIFO

FIFO stands for "First-In, First-Out" and is an inventory management method that assumes that the first items received into an inventory are the first to be sold or used. This means that the oldest inventory items are used or sold first, while the newer items are used or sold later. FIFO is commonly used for products that have a limited shelf life or are perishable, such as food or medicine, to ensure that the older items are used first and do not expire. It can also help businesses manage product obsolescence by ensuring that older products are sold or used before newer ones.

2.4.2 LIFO

LIFO stands for "Last-In, First-Out" and is an inventory management method that assumes that the last items received into an inventory are the first to be sold or used. This means that the newest inventory items are used or sold first, while the older items are used or sold later. LIFO is commonly used for products that have a long shelf life and are not likely to expire or spoil, such as furniture or electronics. It can also help businesses manage rising costs by using the most recently purchased and typically more expensive inventory items first, which can result in lower taxes and higher profits. However, LIFO may not be suitable for businesses that need to track inventory based on expiration dates or other specific characteristics.

2.4.3 FEFO

FEFO stands for "First-Expired, First-Out" and is an inventory management method that assumes that the first products that will expire are the first to be sold or used. This method is commonly used for products that have a limited shelf life or are perishable, such as food, medicine, or cosmetics. By using the FEFO method, businesses can ensure that they use or sell the products that are closest to their expiration dates first, reducing the risk of spoilage or waste.

3.1 System model

3.1.1 Coordinate

To accurately locate the position of a slot, it is essential to establish a coordinate system. The coordinate system provides a framework for identifying the exact location of each slot within the warehouse space. By using a coordinate system, it is possible to implement a more efficient and organized storage system that can help streamline warehouse operations and optimize inventory management:

$$X(i) = \text{[Floor][Section][Location][Shelf][Column][Row]} \tag{3.1}$$

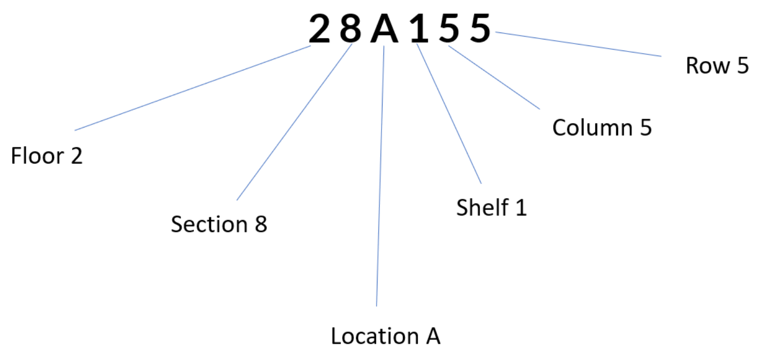


Figure 3.1. A slot in section 8, location A

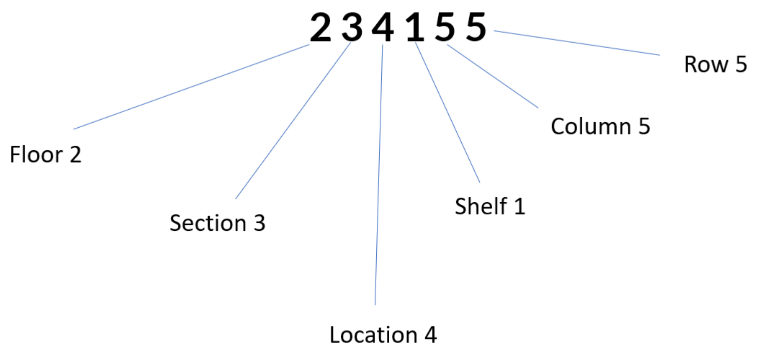


Figure 3.2. A slot in section 3, location 4

The coordinate slot range from 11111 to 38D455 is established with limits and exception (as shown in table 4.1.). These limits will provide a clear framework for locating and organizing items within the warehouse, which will help to improve efficiency and productivity.

Table 3.1. Coordinates

Type	Minimum	Maximum	Exception
Floor	1	3	
Section	1	8	
Location	1	7	A,B,C,D (Section 8)
Shelf	1	4	
Column	1	5	
Row	1	5	

3.1.2 Layout

3.1.2.1 Slot

The following properties of both the slot and the item stored in the slot should be taken into consideration.

Slot coordinate:

$$X_s(i) = \text{[Floor][Section][Location][Shelf][Column][Row]} \tag{3.2}$$

Preferred item identification code to be stored:

$$I_p(i) = \text{[item code]} \tag{3.3}$$

Denote if the slot is empty or not:

$$E_s(i) = \begin{cases} 0, & \text{if there is item stored} \\ 1, & \text{if there the slot is empty} \end{cases} \tag{3.4}$$

Stored item identification code:

$$I_s(i) = \begin{cases} \text{[item code]}, & \text{if } E_s(i) = 0 \\ 0, & \text{if } E_s(i) = 1 \end{cases} \tag{3.5}$$

Stored carrier (carton box) identification code:

$$I_{sc}(i) = \begin{cases} \text{[carrier code]}, & \text{if } E_s(i) = 0 \\ 0, & \text{if } E_s(i) = 1 \end{cases} \tag{3.6}$$

The timestamp when the item is stored:

$$T_s(i) = \text{[received timestamp]} \tag{3.7}$$

Denote if the product is already reserved for order:

$$R_d(i) = \begin{cases} 1, & \text{if stored item is reserved} \\ 0, & \text{if stored item is not reserved} \end{cases} \tag{3.8}$$

Order identification code:

$$I_d(i) = \begin{cases} \text{[order code]}, & \text{if } R_d(i) = 1 \\ \text{[null]}, & \text{if } R_d(i) = 0 \end{cases} \tag{3.9}$$

The estimate despatch timestamp, day, month, year: $T_d(i), D_d(i), M_d(i), Y_d(i)$
Item shelf age at the date of calculation:

$$T_a(i) = \text{ [now timestamp]} - T_s(i) \tag{3.10}$$

3.1.2.2 Column

Column coordinate:

$$X_c(i) = \text{ [Floor][Section][Location][Shelf][Column]} \tag{3.11}$$

Preferred item identification code to be stored:

$$I_c(i) = \text{ [item code]} \tag{3.12}$$

Column storage capacity by slot:

$$C_c(i) = 5 - \sum_{j=1}^{5} E_s(f[X_c(i), j]) \tag{3.13}$$

Column age by the oldest product shelf age:

$$T_c(i) = max[T_a(f[X_c(i)])] \tag{3.14}$$

3.1.3 Parameters

It is important to ensure that the warehouse does not operate at full capacity at all times, as this can lead to potential issues in the event of an emergency. Therefore, we recommend designating specific areas within the warehouse as emergency storage locations. In this case, we suggest assigning locations A, B, C, and D as emergency storage areas, along with the slots within those areas. By designating these specific locations as emergency storage, we can ensure that there is always enough space available to accommodate unexpected inventory or other urgent needs. This will help to ensure that the warehouse can operate safely and efficiently, even in the event of an emergency.

Total slots: $S_t = 18000$

Available slots for allocate: $S_a = 16800$

Emergency slots: $S_e = 1200 = \frac{1}{15}S_t$

Total number of product: $W = 434$

3.1.4 Product

Item code: $I(i)$

Number of product per carrier: $U(i)$

Price of each product: $P_e(i)$

Price of each carrier:

$$P(i) = P_e(i) \times U(i) \tag{3.15}$$

Cost of each product: $A_e(i)$

Cost of each carrier:

$$A(i) = A_e(i) \times U(i) \tag{3.16}$$

Product profit per carrier:

$$B(i) = P(i) - A(i) \tag{3.17}$$

3.1.4.1 Statistic

Total number of stored carrier of the product in the warehouse:

$$Q_s(i) = \sum_{j=1}^{S_t} \begin{cases} 1 - E_s(j), & \text{if } I(i) = I_s(j) \\ 0, & \text{if } I(i) \neq I_s(j) \end{cases} \tag{3.18}$$

Total number of reserved carrier of the product in the warehouse:

$$Q_d(i) = \sum_{j=1}^{S_t} \begin{cases} R_d(j), & \text{if } I(i) = I_s(j) \\ 0, & \text{if } I(i) \neq I_s(j) \end{cases} \tag{3.19}$$

Total number of available carrier of the product in the warehouse:

$$Q_a(i) = Q_s(i) - Q_d(i) \tag{3.20}$$

Total number of carrier staying in emergency slot:

$$Q_e(i) = \sum_{emc} \begin{cases} 1 - E_s(emc), & \text{if } I(i) = I_s(emc) \\ 0, & \text{if } I(i) \neq I_s(emc) \end{cases} \tag{3.21}$$

Oldest available carrier coordinate:

$$X_o(i) = X(j), \quad \text{if} \begin{cases} I(i) = I_s(j) \\ R_d(j) = 0 \\ T_a(j) = max[T_a(f[I(i)])] \end{cases} \tag{3.22}$$

Newest stored carrier coordinate:

$$X_t(i) = X(j), \quad \text{if} \begin{cases} I(i) = I_s(j) \\ R_d(j) = 0 \\ T_s(j) = min[T_s(f[I(i)])] \end{cases} \tag{3.23}$$

3.1.4.2 Record

Number of days each year: $Z_{Y_d(j)}$ (days) <example: $Z_{2022} = 365$ days)>

Total number of days since recorded:

$$Z_r = \sum Z_{Y_d(j)} \text{ (days)} \tag{3.24}$$

Number of orders product appear on each year: $O_{Y_d(j)}(i)$ (orders)

Total number of orders involved since recorded:

$$O_r(i) = \sum O_{Y_d(j)}(i) \text{ (orders)} \tag{3.25}$$

Number of days product appear on order each year: $D_{Y_d(j)}(i)$ (days)

Total number of days involved since recorded:

$$D_r(i) = \sum D_{Y_d(j)}(i) \text{ (days)} \tag{3.26}$$

The number of days involved will not exceed the number of orders involved:

$$D_{Y_d(j)}(i) \leq O_{Y_d(j)}(i) \tag{3.27}$$

$$D_r(i) \leq O_r(i) \tag{3.28}$$

Number of carrier despatched each year: $Q_{Y_d(i)}(i)$ (carriers)

Total number of carrier despatched recorded:

$$Q_r(i) = \sum Q_{Y_d(j)}(i) \text{ (carriers)} \tag{3.29}$$

It is important to note that:

$$\sum_{i=1}^{W} Q_{Y_d(j)}(i) \leq S_t \times Z_{Y_d(j)} \tag{3.30}$$

$$\sum_{i=1}^{W} Q_{r(j)}(i) \leq S_t \times Z_r \tag{3.31}$$

The frequency, or ratio of appearance of the product:

$$F_{Y_d(j)}(i) = \frac{D_{Y_d(j)}(i)}{Z_{Y_d(j)}} \tag{3.32}$$

$$F_r(i) = \frac{D_r(i)}{Z_r} \tag{3.33}$$

The average flow of product:

$$G_{Y_d(j)}(i) = \frac{Q_{Y_d(j)}(i)}{O_{Y_d(j)}(i)} \text{ (carrier per order)} \tag{3.34}$$

$$G_r(i) = \frac{Q_r(i)}{O_r(i)} \text{ (carrier per order)} \tag{3.35}$$

$$g_{Y_d(j)}(i) = \frac{Q_{Y_d(j)}(i)}{D_{Y_d(j)}(i)} \text{ (carrier per day)} \tag{3.36}$$

$$g_r(i) = \frac{Q_r(i)}{D_r(i)} \text{ (carrier per day)} \tag{3.37}$$

Combined with equation (3.27) and (3.28):

$$g_{Y_d(j)}(i) \geq G_{Y_d(j)}(i) \tag{3.38}$$

$$g_{r(j)}(i) \geq G_{r(j)}(i) \tag{3.39}$$

The average slot demand of product:

$$H_{Y_d(j)}(i) = \frac{Q_{Y_d(j)}(i)}{Z_{Y_d(j)}} \text{ (slot per day)} \tag{3.40}$$

$$H_r(i) = \frac{Q_r(i)}{Z_r} \text{ (slot per day)} \tag{3.41}$$

3.2 Control objective

3.2.1 Space allocation

Total average slot demand of all products:

$$V_{Y_d(j)} = \sum_{k=1}^{W} H_{Y_d(j)}(k) = \sum_{k=1}^{W} \frac{Q_{Y_d(j)}(i)}{Z_{Y_d(j)}} \leq S_t \tag{3.42}$$

$$V_r = \sum_{k=1}^{W} H_r(k) = \sum_{k=1}^{W} \frac{Q_r(i)}{Z_r} \leq S_t \tag{3.43}$$

Total stored slot in the warehouse at the current date:

$$V_s = \sum_{j=1}^{S_a} E_s(j) \leq S_a \tag{3.44}$$

Total empty slot in the warehouse at the current date:

$$V_e = S_a - V_s \tag{3.45}$$

Total emergency slot in use at the current date:

$$V_{emc} = \sum E_s(f[X_{emc}]) \leq S_e \tag{3.46}$$

Total empty column in the warehouse at the current date:

$$K_e = \sum_{i=1}^{3360} \begin{cases} 1, & \text{if } C_c(i) = 5 \\ 0, & \text{if } C_c(i) < 5 \end{cases} \quad \text{also } K_e \leq \frac{V_e}{5} \tag{3.47}$$

3.2.1.1 First cycle

In case $V_s = 0$

All time value of each product:

$$N_r(i) = Q_r(i) \times B(i) \tag{3.48}$$

All time value of all products:

$$N_r = \sum_{i=1}^{W} N_r(i) \tag{3.49}$$

Percentage of all time value for each product:

$$L_r(i) = \frac{N_r(i)}{N_r} \tag{3.50}$$

In accord to the annual percentage $L_r(i)$ and frequency $F_r(i)$ of each product, which is sorted from high to low. Applying ABC analysis method, the number of columns allocate to each product can be calculated.

$$K(i) = \begin{cases} \frac{max[g_r(i),H_r(i)]}{5}, & \text{case A} \\ \frac{max[G_r(i),H_r(i)]}{5}, & \text{case B} \\ \frac{min[G_r(i),H_r(i)]}{5}, & \text{case C} \end{cases} \tag{3.51}$$

As long as the following condition is sastified:

$$\sum_{i=1}^{W} K(i) \leq \frac{S_a}{5} \tag{3.52}$$

Further elaborate with equation (3.35) and (3.43) in case C of equation (3.51):

$$\sum_{i=1}^{W} min[G_r(i),H_r(i)] \leq \sum_{k=1}^{W} H_{Y_r}(i) \leq S_t \tag{3.53}$$

Also statistic report in 2019 (appendix A) show that:

$$\sum_{i=1}^{W} U(i) \times Q_{2019}(i) = 75982216, \text{ with } U_{avg} \approx 458 \tag{3.54}$$

Hence average slot demand of all products in 2019:

$$V_{2019} = \sum_{i=1}^{W} \frac{Q_{2019}(i)}{Z_{2019}} \approx 165900 \approx \frac{1}{39,6} \times S_t \tag{3.55}$$

3.2.1.2 Annual cycle

Previous year:

$$Y_p = 2022 \text{ (sample)} \tag{3.56}$$

Annual value of each product:

$$N_{Y_p}(i) = Q_{Y_p}(i) \times B(i) \tag{3.57}$$

Annual value of previous year:

$$N_{Y_p} = \sum_{i=1}^{W} N_{Y_p}(i) \tag{3.58}$$

Percentage of annual value for each product:

$$L_{Y_p}(i) = \frac{N_{Y_p}(i)}{N_{Y_p}} \tag{3.59}$$

In accord to the annual percentage $L_{Y_p}(i)$ and frequency $F_{Y_p}(i)$ of each product, which is sorted from high to low. Applying ABC analysis method, the number of columns allocate to each product can be calculated.

$$K_a(i) = \begin{cases} K_e \times \frac{max[g_{Y_p}(i), H_{Y_p}(i)]}{S_a}, & \text{case A} \\ K_e \times \frac{max[G_{Y_p}(i), H_{Y_p}(i)]}{S_a}, & \text{case B} \\ K_e \times \frac{min[G_{Y_p}(i), H_{Y_p}(i)]}{S_a}, & \text{case C} \end{cases} \tag{3.60}$$

As long as the following condition is sastified:

$$\sum_{i=1}^{W} K_a(i) \leq K_e \tag{3.61}$$

It is worth noting that the annual cycle can be adjusted to a quarterly or monthly cycle to allow for more flexible space allocation. By adjusting the cycle to a shorter timeframe, it becomes easier to adjust warehouse operations based on changes in inventory needs and other factors. This can help to optimize space allocation and improve overall warehouse efficiency.

3.2.2 Receiving and put-away

Although TPP plastic products have a lifespan of about 50 years, it is still important to have a storage strategy in place to prevent products from sitting on the shelf for too long. Therefore, a FIFO (first in, first out) strategy has been chosen to ensure that older products are used or sold before newer ones.

Based on the FIFO strategy, the following variables are to be considered when the system receives a new carrier: product identification code I and the corresponding product newest stored coordinate X_t.

Find youngest column with suitable capacity (there is empty slot within):

$$X = \begin{cases} f[X_c(f[X_t])], & \text{if } C_c(f[X_t]) > 0 \\ f[X_c(j)], & \text{if } \begin{cases} C_c(f[X_t]) = 0 \\ T_c(j) = min[T_c(f[I])] \\ C_c(j) > 0 \end{cases} \end{cases} \tag{3.62}$$

In case there are multiple empty columns as well as full columns that have been designated for a specific product, it is recommend to select the empty column that is closest to the full ones:

$$X = f[X_c(j)], \quad \begin{cases} C_c(f[I]) = 0 \\ C_c(f'[I]) = 5 \\ |j - f[I]| \text{ is minimum} \end{cases} \tag{3.63}$$

3.2.3 Order fulfillment

There are two scenarios: either the total available product carrier $Q_a(i)$ is sufficient to fulfill the order, or it is insufficient.

3.2.3.1 Sufficient

When considering the age column, columns $X_c(i)$ are selected from the oldest to the youngest <$T_c(i)$>. In each column, carriers $X(i)$ are selected using the same principle <$T_a(i)$>.

Hence, there is no emergency request send to the factory:

$$EMC(i) = 0 \tag{3.64}$$

3.2.3.2 Insufficient

Place a replenishment request for the emergency locations. Carriers in here should be placed order by order, with priority given to the first order to be picked.

$$EMC(i) = order\ numbers\ (i) - Q_a(i) \tag{3.65}$$

Provided that the following condition is met:

$$\sum EMC_{D_d} \leq factory\ capacity \tag{3.66}$$

3.2.4 Daily replenishment

To calculate the daily replenishment for the warehouse, both the factory capacity and emergency requests need to be taken into consideration.

$$daily\ replenishment\ quota = factory\ capacity - \sum EMCD_d \tag{3.67}$$

Based on the product frequency from the previous year F_{Y_d}, products should be picked in order of higher frequency, while ensuring that the total replenishment does not exceed the quota.

$$\sum C_c(i) = \sum(5 \times K_a(i)) + \sum C_{c,prev}(i) \leq \ daily\ replenishment\ quota \tag{3.68}$$

4.1 Control algorithm

4.1.1 Space allocation

In the control loop shown in Figure 4.1., the input are the product statistics derived from equations (3.26), (3.29), (3.10), (3.16), (3.15) and (3.17); the output is a plan task to allocating space. The controller uses the information of each product, to generate a control signal that consists of two components: the number of columns assigned to the product and priority values used to allocate space determines the output.

Control loop for space allocation cycle:

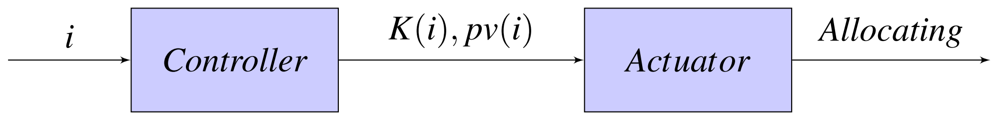


Figure 4.1. Space allocation control loop block diagram

4.1.2 Daily replenishment

In the control loop depicted in Figure 4.2., the input, or set point, is the total available slots for allocation. The feedback signal is the total number of stored slots (excluding emergency slots), which is calculated using equation (3.44). The output is the daily replenishment request sent to the factory. The disturbance is the emergency request. The error signal is the total number of empty slots (excluding emergency slots), calculated using equation (3.45). The actuating signal is the storage capacity of columns, calculated using equation (3.13). The controller uses the product frequency to prioritize which product to replenish first.

Replenishment control loop:

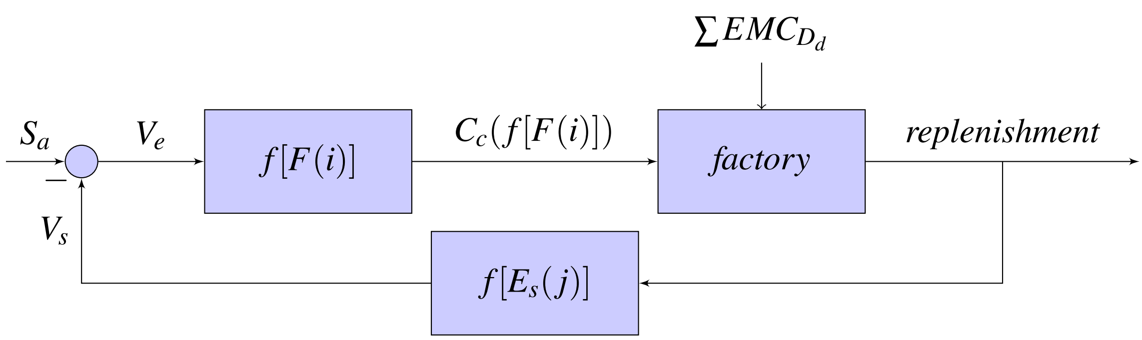


Figure 4.2. Replenishment control loop block diagram

4.1.3 *Receiving and put-away*

Based on Figure 4.3., the algorithm takes information about the new carrier as input and outputs the coordinate where the system will place it. The algorithm finds the most suitable slot that satisfies the FIFO strategy from the column that stores or is assigned to the same item.

A control algorithm for the receiving and put-away process was designed using equations (3.62) and (3.63):

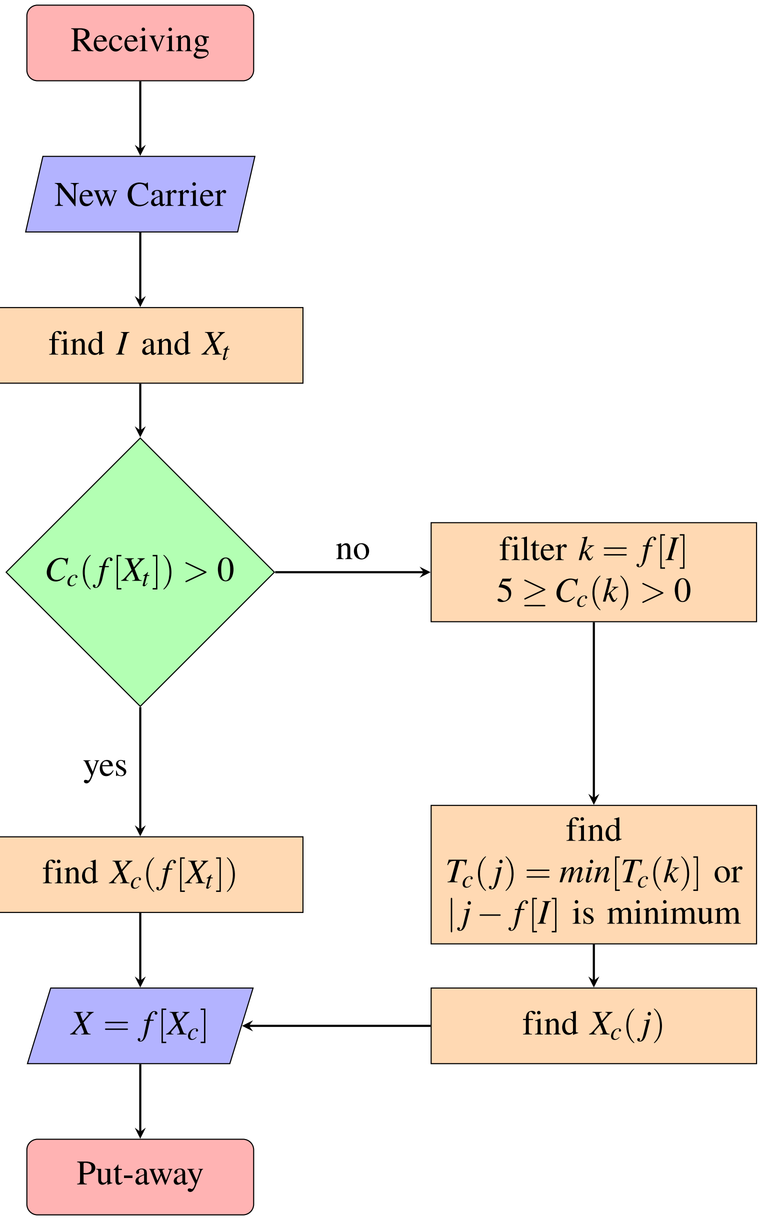


Figure 4.3. Receiving and put-away flowchart diagram

4.1.4 *Order fulfillment*

Based on the flowchart diagram for order fulfillment shown in Figure 4.4., the algorithm takes the product identification code from a new order as its input and outputs the locations where the system will pick the products. The algorithm finds the most suitable slot that satisfies the FIFO strategy and requests more products from the factory in cases where there are insufficient numbers.

A control algorithm for order fulfillment process is designed based on equations (3.64), (3.65), and (3.66):

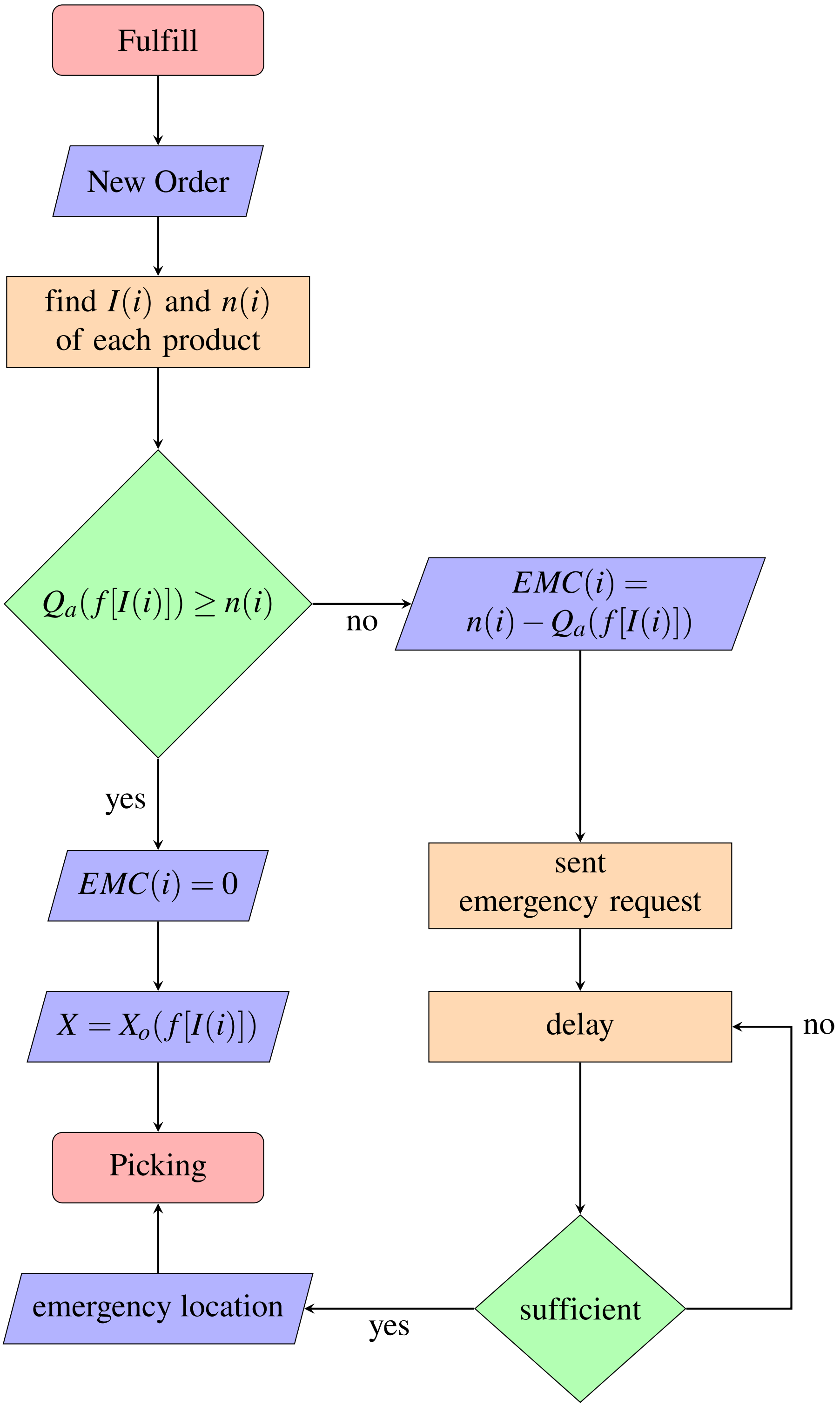


Figure 4.4. Order fulfillment flowchart diagram

4.2 Fuzzy Logic Controller

Considering the nonlinear relationship between the product information and the desired space allocation, as well as the complexity of handling a MIMO system with 6 inputs and 2 outputs, a Fuzzy Logic Controller (FLC) was proposed to meet the space allocation control requirement. FLC offer a valuable approach to address the uncertainties and inherent imprecision present in warehouse management. These environments involve variables such as product characteristics, storage requirements, and available space that can exhibit variations and vagueness. FLC excel in handling such complexities by leveraging fuzzy logic principles.

4.2.1 Define the variables

Fuzzy logic empowers the FLC to effectively handle a wide range of input attributes, including factors such as product sales, manufacturing cost, price tags, and other pertinent variables. By embracing the fuzzy nature of these attributes, the FLC can make well-informed decisions when it comes to the optimal allocation of space for different product types. The six inputs should be:

- Number of carrier.
- Shelf age.
- Cost.
- Price.
- Profit.
- Frequency.

The two outputs:

- Assign parameter.
- Priority value.

4.2.2 Membership functions

To represent the fuzzy sets for each variables, appropriate membership functions can be defined using the Mamdani approach. These functions facilitate the mapping of crisp input values to fuzzy values, incorporating linguistic terms such as "low," "medium," or "high" to express the degree of membership.

In this case, it is crucial to consider the parameters for each membership function after meticulously training the model using appropriate data.

To ensure the effectiveness and accuracy of the model, it is necessary to train it with relevant data that accurately represents the relationships and characteristics of the problem at hand. This training process involves determining the optimal values for the parameters associated with each membership function.

4.2.3 Fuzzy rules

To guide space allocation decisions based on fuzzy values, it is essential to establish a well-defined set of fuzzy rules that capture the relationships between the input and output variables. These rules serve as the logic framework for the Fuzzy Logic Controller. Consulting with experts and conducting careful data analysis is recommended during this rule establishment process.

When determining the number of rules, a careful balance needs to be struck. If too few rules are employed, the controller system may become too sparse, resulting in incomplete decision-making. On the other hand, if an excessive number of rules are used, the controller system may become too complex and may struggle to execute efficiently. To strike the right balance, it is important to consider the complexity of the problem, the available data, and expert knowledge. Through a systematic approach, rules can be crafted to capture the essential relationships and decision-making logic while avoiding unnecessary redundancy.

Expert advice and domain knowledge play a crucial role in determining the appropriate number of rules. Additionally, careful analysis of relevant data helps identify patterns and dependencies that can inform the rule creation process.

Table 4.1. A general rules sample to assign parameter

Value	NoC	Age	Cost	Price	Profit	Frequency
High	high	low	low	medium	high	high
Medium	high	low	low	medium	high	medium
Low	low	high	high	low	low	low

4.2.4 Fuzzy inference

To determine the appropriate output values based on the fuzzy rules and input values, fuzzy inference methods such as the Mamdani approach will be applied. This process encompasses aggregating and combining the fuzzy values derived from the rules and input variables, followed by defuzzification to obtain crisp output values.

By utilizing fuzzy inference, the Fuzzy Logic Controller can effectively interpret the fuzzy rules and input values to generate meaningful output values. Aggregation and combination techniques, such as the max-min method in the Mamdani approach, are employed to synthesize the fuzzy values and derive a consolidated result.

After the aggregation process, defuzzification is performed to convert the fuzzy output values into a crisp output value that can be readily comprehended and utilized. Defuzzification methods like centroid, mean of maximum, or weighted average are commonly employed to obtain a single, representative crisp output value.

Once this process is completed, the resulting output from the controller becomes suitable for the user to commence space allocation tasks. The crisp output value provides clear instructions for allocating space within the warehouse, facilitating efficient and effective utilization of available resources.

By applying fuzzy inference methods, aggregating and combining fuzzy values, and subsequently defuzzifying to obtain crisp output values, the Fuzzy Logic Controller enables users to make informed decisions and initiate space allocation activities with confidence.

4.2.5 Controller optimization

Fine-tune the FLC parameters and rules to optimize the space allocation process. This can involve expert knowledge, data analysis, or even using machine learning techniques to improve the FLC's performance.

4.3 System architecture

4.3.1 Architecture

A n-tier architecture is well-suited for a WMS due to its scalability, maintainability, security, and flexibility. It allows you to scale different parts of the system independently, make it easier to maintain and update the different layers, improve the security of the system, and give you more flexibility to choose different technologies or frameworks for each layer. [9]

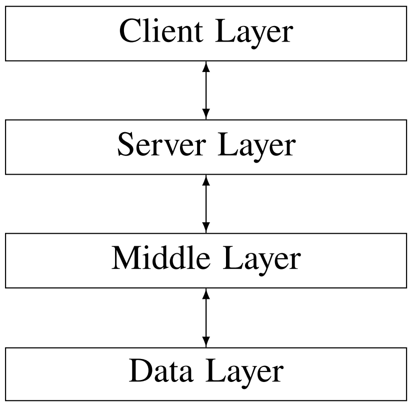


Figure 4.5. 4-tier architecture

4.3.2 Structure

The MVT (Model-View-Template) architecture (Figure 4.6.) is a design pattern that separates the concerns of a software application into three components: the Model, the View, and the Template. This architecture can be applied to various systems, including databases, user interfaces (UI), and static templates for WMS.

In the MVT architecture, the Model represents the data and the control logic of the application. It encapsulates the storage and retrieval of data from the database, as well as any operations or calculations performed on the data.

The View component handles the user interface and interacts with the users.

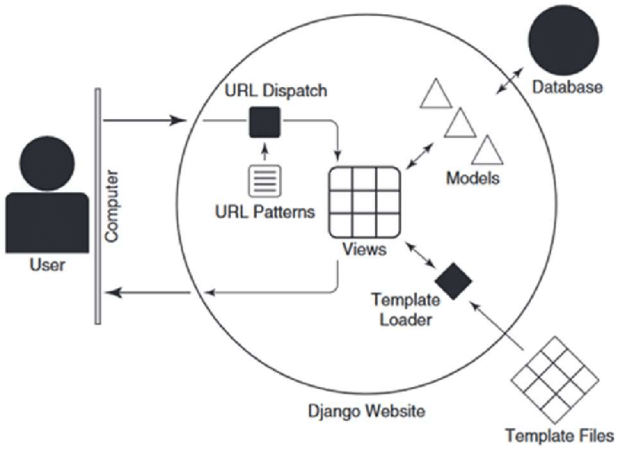


Figure 4.6. Model - View - Template architecture

It presents the data from the Model to the user and gathers input from the user to perform actions or update the data.

The Template component is responsible for defining the structure and layout of the UI. It specifies how the data from the Model should be displayed to the user. Templates can be static HTML files or dynamic templates that incorporate logic to generate the UI based on the data.

4.3.3 Tasks

A task queue system enables servers to delegate time-consuming or resource-intensive tasks to be executed asynchronously in the background. To facilitate this functionality, the task queue system relies on a message broker to handle communication between the server application and the worker processes responsible for task execution.

Configuring the broker settings establish a connection between the task queue system and the message broker, enabling the utilization of its capabilities for distributed task processing or messaging within the server application.

4.4 System flow

4.4.1 User login

The sequence diagram Figure 4.7. shows the process of a user logging into a system with different layers of the architecture interacting with each other. The sequence starts with the user entering their account and password information in the Client Layer. The Client Layer sends this information to the Server Layer, which validates the user's credentials and sends an access response. The Server Layer then sends a validation request to the Middle Layer, which retrieves the user's role from the Data Layer and performs a validation process to ensure that the user has the necessary permissions to access the requested resources. Once the validation is complete, the Middle Layer sends a session response to the Server Layer, which in turn sends the response to the Client Layer indicating that the user is authenticated and can access

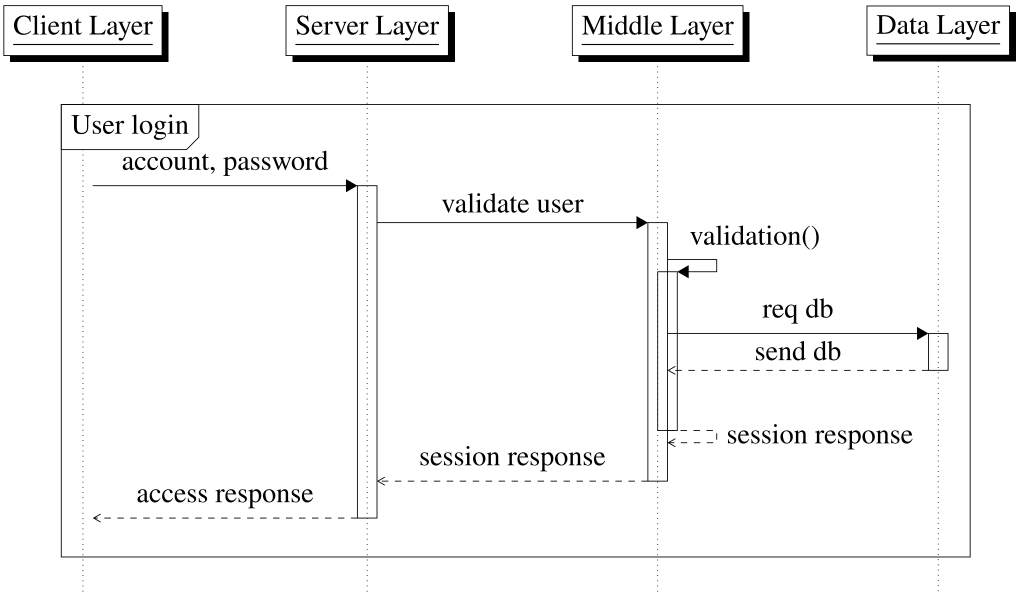


Figure 4.7. Login sequence diagram

the system.

4.4.2 Receiving and put-away

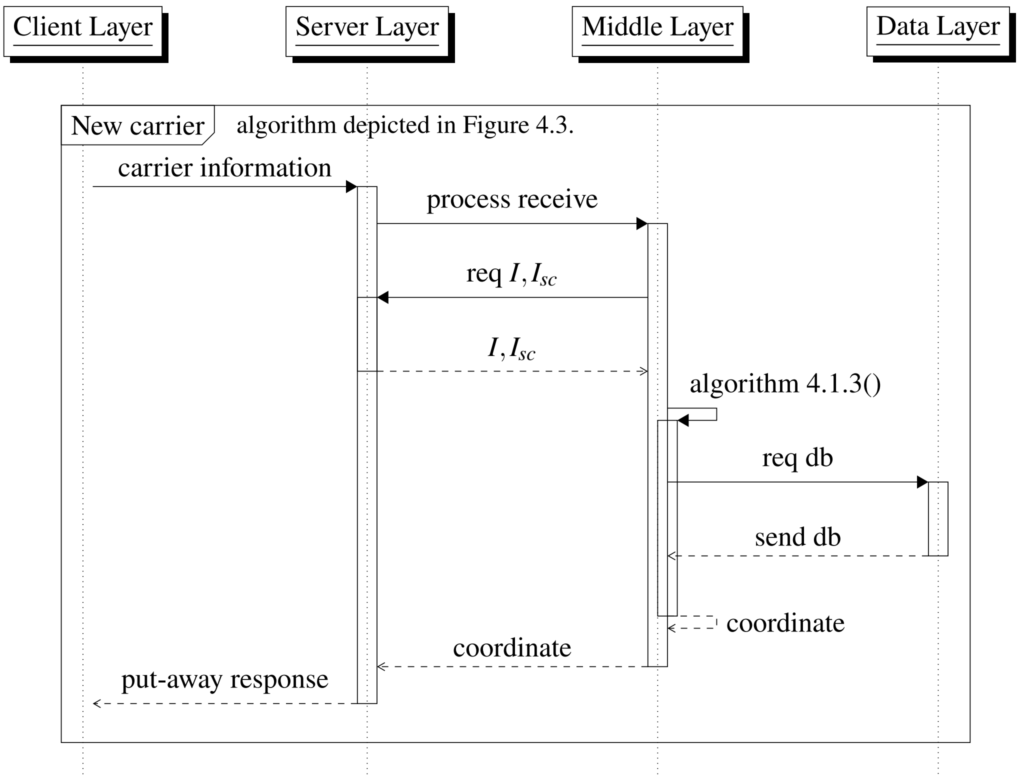


Figure 4.8. Receiving and put-away sequence diagram

The sequence diagram Figure 4.8. depicts the process of adding a new carrier to a WMS. The sequence starts with the Client Layer sending carrier information to the Server Layer. The Server Layer processes the request by calling the "receive" function in the Middle Layer. The Middle Layer coordinates the request and calls the "receiving and put-away" algorithm, which updates the databases with the new carrier information and sends a coordinate back to the Server Layer. The Server Layer then sends a put-away response to the Client Layer indicating that the carrier has been successfully integrated to the system.

4.4.3 Order fulfillment

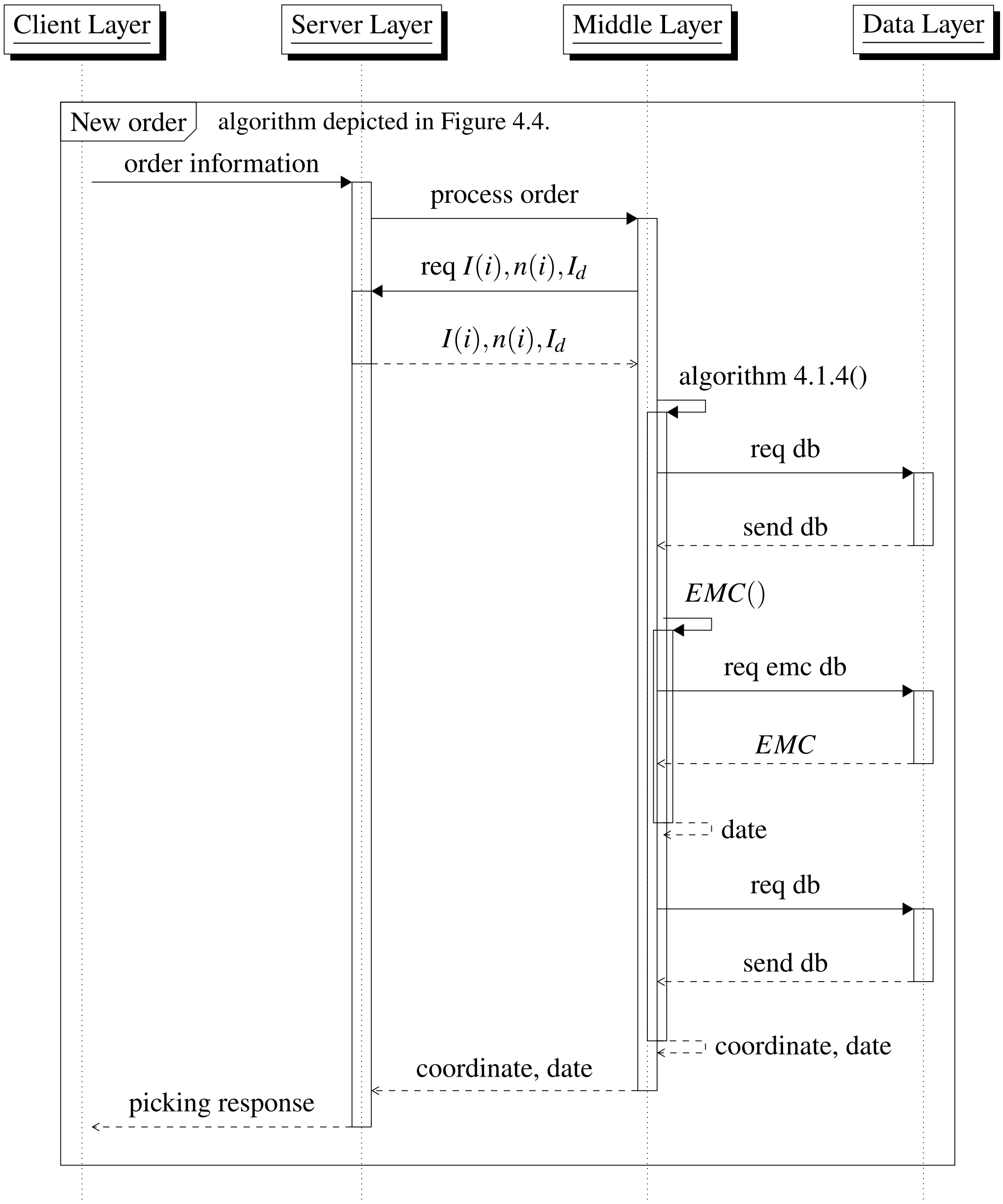


Figure 4.9. Order fulfillment sequence diagram

The sequence diagram Figure 4.9. shows the process of fulfilling a new order in a warehouse management system. The sequence starts with the Client Layer sending order information to the Server Layer. The Server Layer processes the request by calling the "order" function in the Middle Layer. The Middle Layer coordinates the request and calls the "order fulfillment" algorithm, which updates the databases with the order information and sends a coordinate and date response back to the Server.

The Server then sends a picking response to the Client Layer indicating that the order can be fulfilled.

4.4.4 Daily replenishment

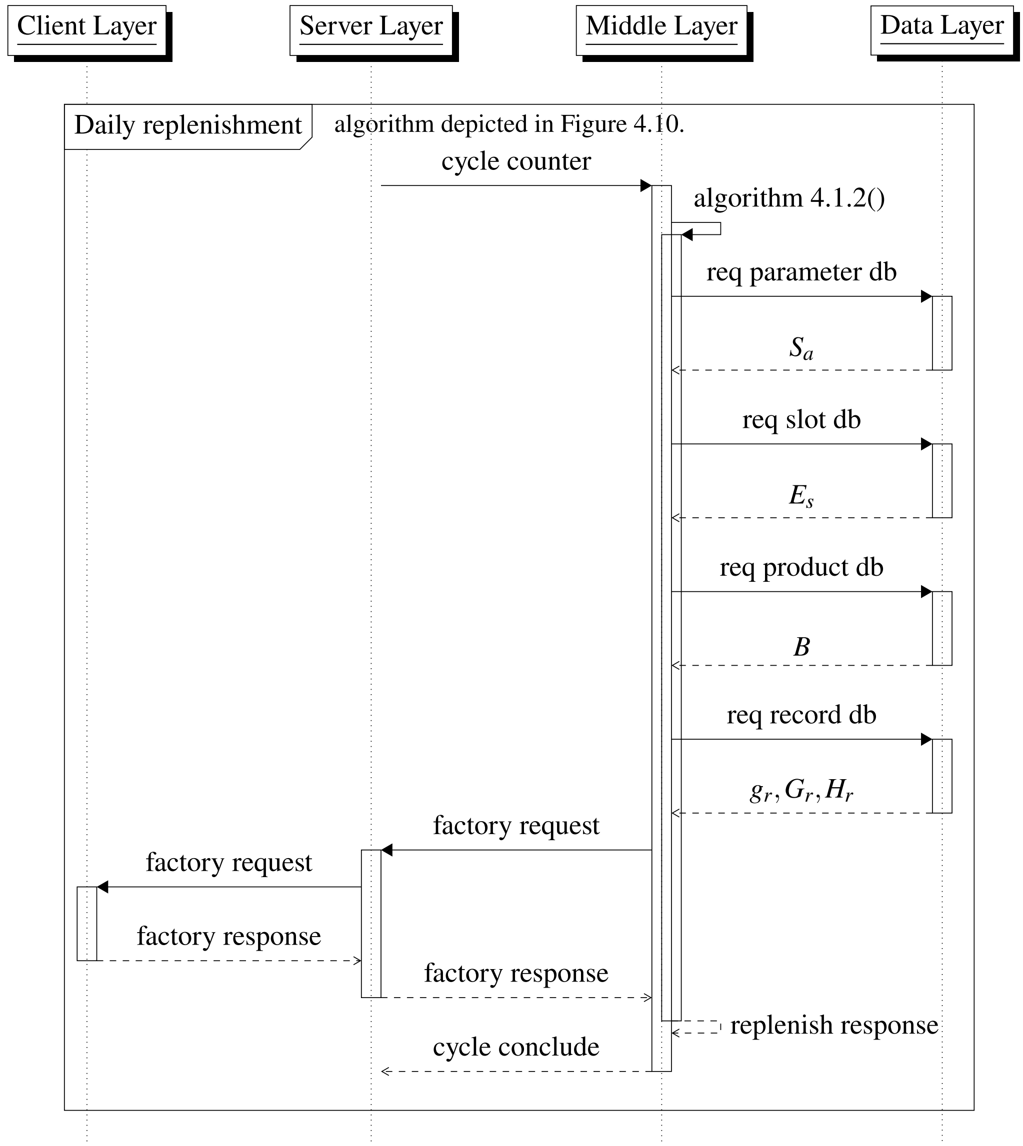


Figure 4.10. Daily replenishment sequence diagram

The sequence diagram Figure 4.10. illustrates the daily replenishment process in a warehouse management system. The sequence begins when the Server Layer sends a cycle counter to the Middle Layer, which then calls the "daily replenishment" algorithm. The algorithm interacts with several databases in the Data Layer, such as the parameter database, slot database, product database, and record database. The algorithm also sends a factory request to the Server Layer, which sends the request to the Client Layer to obtain the necessary products. Once the factory response is received, the Middle Layer send a replenish response to conclude the cycle.

4.4.5 Space allocation

The sequence diagram in Figure 4.11. illustrates the process of space allocation in a warehouse management system. The sequence begins with the Server Layer sending a cycle counter to the Middle Layer, which then calls the "space allocation" algorithm. The algorithm interacts with several databases in the Data Layer to output a quota for each product. The algorithm then calls the "allocate" function, using the quota to allocate space for each product. The Middle Layer then sends the allocation information to the Data Layer, which updates the slot database. Finally, the cycle concludes.

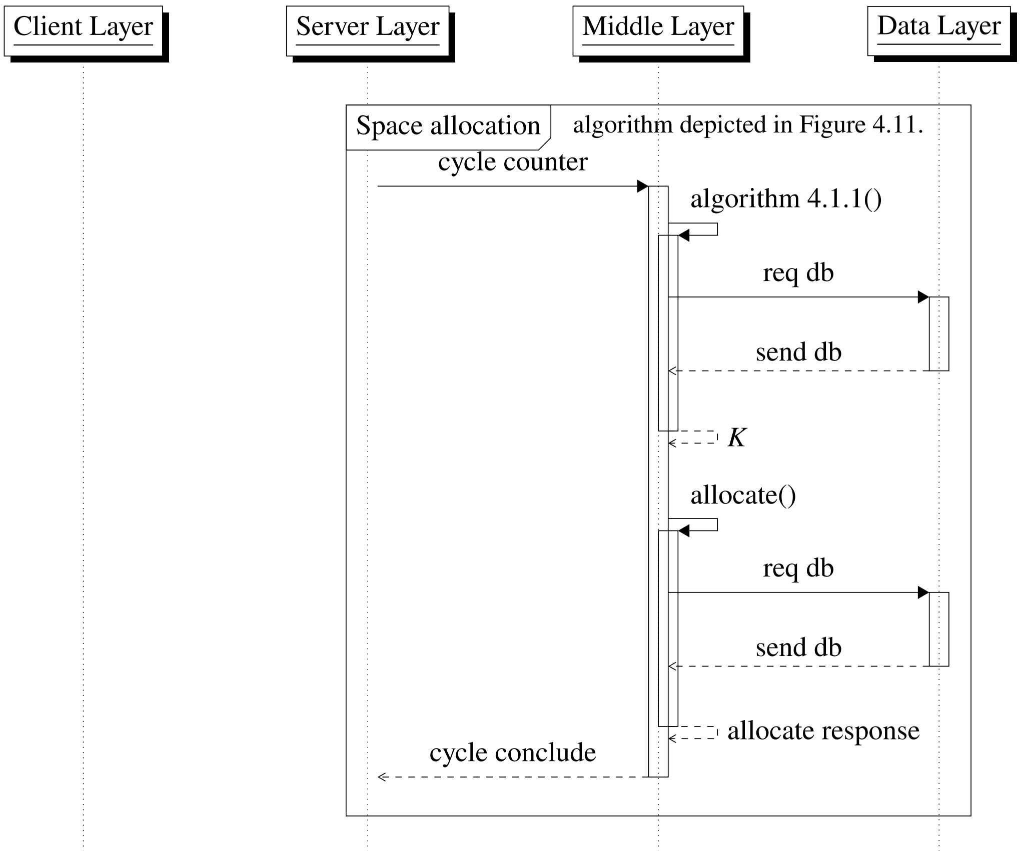


Figure 4.11. Space allocation sequence diagram

5.1 Platform

After carefully considering the project requirements, team capabilities, and current trends, the decision has been made to adopt the Django web framework and its Model-View-Template (MVT) architecture. This choice aligns well with the project's needs. Alongside Django, the development will involve programming languages such as Python, CSS, HTML, and JavaScript, and the chosen database will be SQLite.

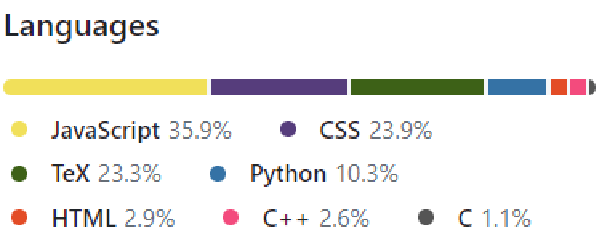


Figure 5.1. Computer languages

To enhance the asynchronous task processing capabilities, the task queue system Celery will be employed in conjunction with the Redis server as the broker. This combination will efficiently handle distributed task processing and messaging within the Django application.

Furthermore, in the middleware tier, C++ and C will be utilized to assist in building the application, providing additional functionality and support for the project.

By leveraging the Django framework, employing Celery with Redis as the broker, and utilizing C/C++ in the middleware tier, the project can benefit from robust web development, efficient task processing, and enhanced application capabilities.

Figure 5.2. Platforms

5.2 Web server

5.2.1 User

Currently, for debugging and testing purposes, having a superuser account with unrestricted access is deemed sufficient. However, the distribution of system access will be developed at a later date to ensure appropriate levels of access and permissions for different user roles or entities.

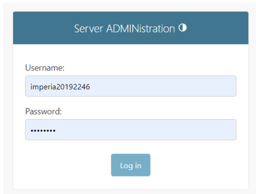


Figure 5.3. Admin login form

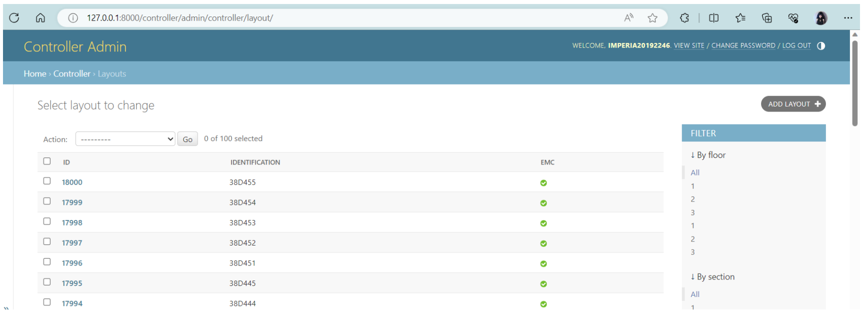


Figure 5.4. Administration dashboard for controller

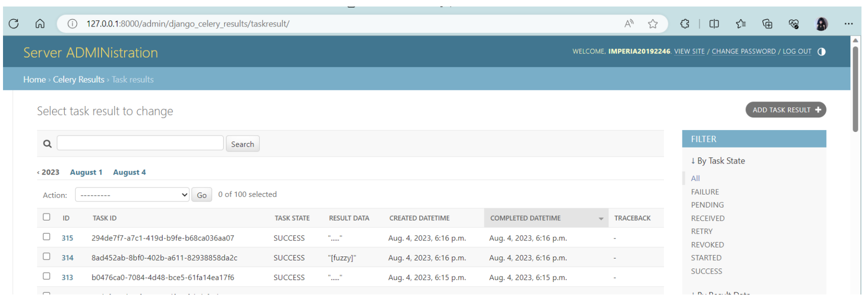


Figure 5.5. Administration dashboard for server

5.2.2 Server

The server has been launched and is operating normally. The connection and access to the database, along with the middleware application, are functioning well.

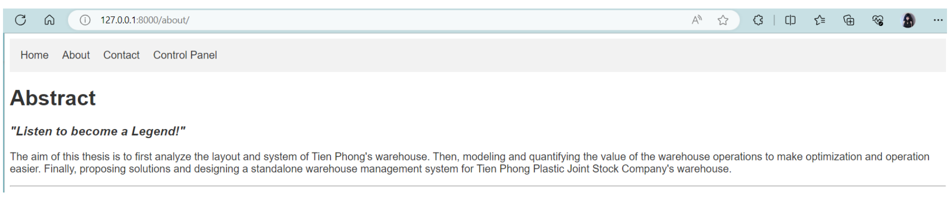
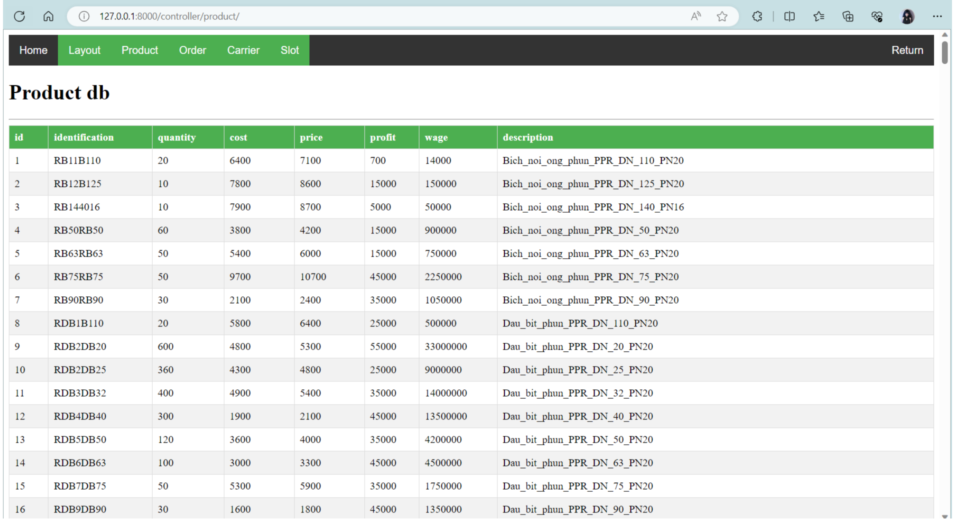


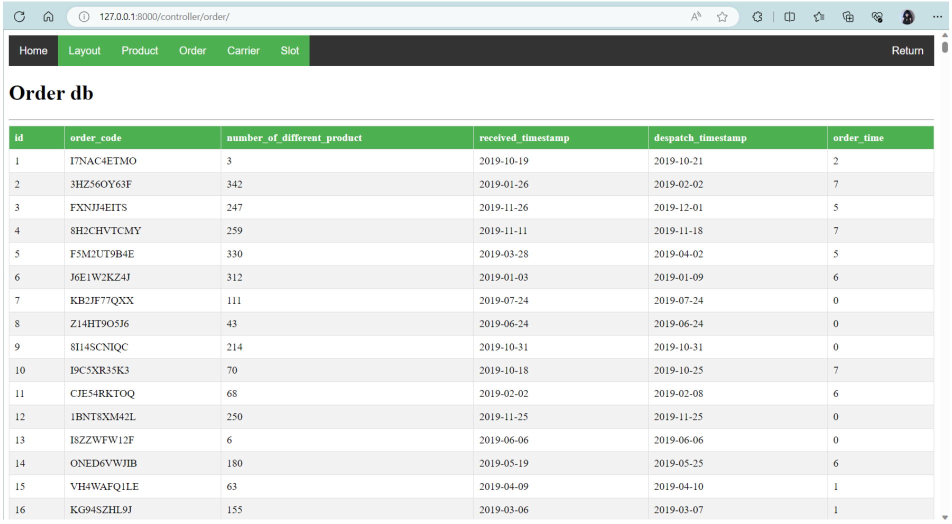
Figure 5.6. Server about page



Product db

id	identification	quantity	cost	price	profit	wage	description
1	RB11B110	20	6400	7100	700	14000	Bich_noi_ong_phun_PPR_DN_110_PN20
2	RB12B125	10	7800	8600	15000	150000	Bich_noi_ong_phun_PPR_DN_125_PN20
3	RB144016	10	7900	8700	5000	50000	Bich_noi_ong_phun_PPR_DN_140_PN16
4	RB50RB50	60	3800	4200	15000	900000	Bich_noi_ong_phun_PPR_DN_50_PN20
5	RB63RB63	50	5400	6000	15000	750000	Bich_noi_ong_phun_PPR_DN_63_PN20
6	RB75RB75	50	9700	10700	45000	2250000	Bich_noi_ong_phun_PPR_DN_75_PN20
7	RB90RB90	30	2100	2400	35000	1050000	Bich_noi_ong_phun_PPR_DN_90_PN20
8	RDB1B110	20	5800	6400	25000	500000	Dau_bit_phun_PPR_DN_110_PN20
9	RDB2DB20	600	4800	5300	55000	33000000	Dau_bit_phun_PPR_DN_20_PN20
10	RDB2DB25	360	4300	4800	25000	9000000	Dau_bit_phun_PPR_DN_25_PN20
11	RDB3DB32	400	4900	5400	35000	14000000	Dau_bit_phun_PPR_DN_32_PN20
12	RDB4DB40	300	1900	2100	45000	13500000	Dau_bit_phun_PPR_DN_40_PN20
13	RDB5DB50	120	3600	4000	35000	4200000	Dau_bit_phun_PPR_DN_50_PN20
14	RDB6DB63	100	3000	3300	45000	4500000	Dau_bit_phun_PPR_DN_63_PN20
15	RDB7DB75	50	5300	5900	35000	1750000	Dau_bit_phun_PPR_DN_75_PN20
16	RDB9DB90	30	1600	1800	45000	1350000	Dau_bit_phun_PPR_DN_90_PN20

Figure 5.7. The controller site showcases the product database



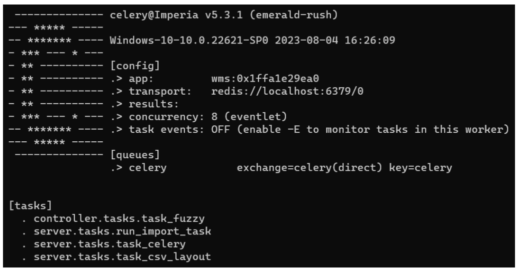
Order db

id	order_code	number_of_different_product	received_timestamp	despatch_timestamp	order_time
1	I7NAC4ETMO	3	2019-10-19	2019-10-21	2
2	3HZ56OY63F	342	2019-01-26	2019-02-02	7
3	FXNJJ4EITS	247	2019-11-26	2019-12-01	5
4	8H2CHVTCMY	259	2019-11-11	2019-11-18	7
5	F5M2UT9B4E	330	2019-03-28	2019-04-02	5
6	J6E1W2KZ4J	312	2019-01-03	2019-01-09	6
7	KB2JF77QXX	111	2019-07-24	2019-07-24	0
8	Z14HT9O5J6	43	2019-06-24	2019-06-24	0
9	8I14SCNIQC	214	2019-10-31	2019-10-31	0
10	I9C5XR35K3	70	2019-10-18	2019-10-25	7
11	CJE54RKTOQ	68	2019-02-02	2019-02-08	6
12	1BNT8XM42L	250	2019-11-25	2019-11-25	0
13	I8ZZWFW12F	6	2019-06-06	2019-06-06	0
14	ONED6VWJIB	180	2019-05-19	2019-05-25	6
15	VH4WAFQ1LE	63	2019-04-09	2019-04-10	1
16	KG94SZHL9J	155	2019-03-06	2019-03-07	1

Figure 5.8. The controller site showcases the order record

5.2.3 *Task queue system*

The task queue system Celery is currently functioning stably and is well-coordinated with the Redis server, which serves as a broker to coordinate workers to complete tasks in parallel.



```
 -------------- celery@Imperia v5.3.1 (emerald-rush)
--- ***** -----
-- ******* ---- Windows-10-10.0.22621-SP0 2023-08-04 16:26:09
- *** --- * ---
- ** ---------- [config]
- ** ---------- .> app:         wms:0x1ffa1e29ea0
- ** ---------- .> transport:   redis://localhost:6379/0
- ** ---------- .> results:
- *** --- * --- .> concurrency: 8 (eventlet)
-- ******* ---- .> task events: OFF (enable -E to monitor tasks in this worker)
--- ***** -----
 -------------- [queues]
                .> celery           exchange=celery(direct) key=celery


[tasks]
  . controller.tasks.task_fuzzy
  . server.tasks.run_import_task
  . server.tasks.task_celery
  . server.tasks.task_csv_layout
```

Figure 5.9. Celery tasks queue system is ready

```
]"
1691150809.373723 [0 127.0.0.1:46270] "EXEC"
1691150809.375202 [0 127.0.0.1:46258] "BRPOP" "celery" "celery\x06\x163" "celery\x06\x166" "celery\x06\x169" "1"
1691150810.388373 [0 127.0.0.1:46258] "BRPOP" "celery" "celery\x06\x163" "celery\x06\x166" "celery\x06\x169" "1"
1691150811.042635 [0 127.0.0.1:46326] "PUBLISH" "/0.celeryev/worker.heartbeat" "{\"body\": \"eyJob3N0bmFtZSI6ICJjZWxlcnlASW1wZXJpYSIsICJ1dGNvZmZzZXQiOiAtNyw
gInBpZCI6IDE0Mzc2LCAiY2xvY2siOiA5NDk4LCAiZnJlcSI6IDIuMCwgImFjdGl2ZSI6IDEsICJwcm9jZXNzZWQiOiA0MDUsICJsb2FkYXZnIjogWzAuMCwgMC4wLCAwLjBdLCAic3dfaWRlbnQiOiAicHk
tY2VsZXJ5IiwgInN3X3ZlciI6ICI1LjMuMSIsICJzd19zeXMiOiAiV2luZG93cyIsICJ0aW1lc3RhbXAiOiAxNjkxMTUwODExLjA0MTAyNiwgInR5cGUiOiAid29ya2VyLWhlYXJ0YmVhdCJ9\", \"conte
nt-encoding\": \"utf-8\", \"content-type\": \"application/json\", \"headers\": {\"hostname\": \"celery@Imperia\"}, \"properties\": {\"delivery_mode\": 1, \"
delivery_info\": {\"exchange\": \"celeryev\", \"routing_key\": \"worker.heartbeat\"}, \"priority\": 0, \"body_encoding\": \"base64\", \"delivery_tag\": \"32
cf0604-d214-4d4c-880d-5696c76a5a77\"}}"
```

Figure 5.10. Redis server show message with celery

5.3 Controller

5.3.1 Data

According to Appendix A, the report shows that in 4 out of 8 sections, the total amount of exported items was 36,982,276 during the first 10 months of 2019. It is estimated that a total of 75,982,216 items were sold for the entire year. Based on this data, the product and layout database were created, and an application was designed to generate data for training the controller model.

Since the data was randomly generated by machine, is is still in a machine order. A total of 2997 orders, 648279 carriers or 648279 items data was generated.

id ↑	order_code	carrier_code	product_code	stored_timestamp	despatch_timestamp	shelf_age
420272	HUJS19JF55	CKZ9OWAOOK7R4H4P0TM8	RT11T110	2019-02-13	2019-02-24	11
420273	HUJS19JF55	7BHBWWSJCI8IARCP8TZ4	RTC77550	2019-02-02	2019-02-24	22
420274	HUJS19JF55	B77QH2QGL0PFAVBN4A0W	RNRTT25X	2019-02-24	2019-02-24	0
420275	HUJS19JF55	Z2H8WQL0QJ55D56QGLEZ	RNRNRN50	2019-01-26	2019-02-24	29
420276	HUJS19JF55	N4WNBBODNCYK5V99J20Z	RV32V32X	2019-02-02	2019-02-24	22
420277	HUJS19JF55	3OIZ9MC209JWUD7VBZPZ	RUNRRT25	2019-01-25	2019-02-24	30
420278	HUJS19JF55	RCJEBRYFHWY0S3ZVJW04	RL25L25X	2019-01-29	2019-02-24	26
420279	HUJS19JF55	2LWUSCU8L3ZSNI8FUNNF	RZRTRT25	2019-01-26	2019-02-24	29
420280	HUJS19JF55	S6KK6X4SQO6FLWFONDGY	RNRNRN20	2019-02-16	2019-02-24	8
420281	HUJS19JF55	FQTTRNY6AAENT2RMEEE8	RT25T25X	2019-02-23	2019-02-24	1
420282	HUJS19JF55	H9FWWFOITEX0IG0E3CY7	RTRNRN25	2019-02-18	2019-02-24	6
420283	HUJS19JF55	3K64V1TLHFKXV7HV7HCZ	RZRNRN20	2019-01-26	2019-02-24	29
420284	HUJS19JF55	JQ49WSCLGWMKMZ1A910B	RZRTRT20	2019-02-08	2019-02-24	16
420285	HUJS19JF55	01SOMY8KZK0FSB2431E1	RUNC5040	2019-02-02	2019-02-24	22
420286	HUJS19JF55	ERSS90GYXOX7SNYD2TT9	RUTRRN25	2019-01-25	2019-02-24	30
420287	HUJS19JF55	AUAFY8RAAI7NWSXOL8J5	RNB1B180	2019-01-31	2019-02-24	24
420288	HUJS19JF55	689MLVH1HLV8EO7YV4OS	RNRNN50X	2019-02-11	2019-02-24	13
420289	HUJS19JF55	GICNRWAI2JNZ8GGLNBX1	RUTC3220	2019-02-07	2019-02-24	17
420290	HUJS19JF55	66R5LC5300CJVQ4CPFIW	RZRNRN25	2019-01-27	2019-02-24	28
420291	HUJS19JF55	AC7TZP9D1APYW1EB91TR	RVC3C32X	2019-01-26	2019-02-24	29
420292	HUJS19JF55	87GHB2UOP0CKXKMCAHES	RNRTRT25	2019-02-10	2019-02-24	14
420293	HUJS19JF55	IJKLNPJK25WDK9XPQDSO	RUL5UL50	2019-02-23	2019-02-24	1
420294	HUJS19JF55	07D57FAY7SXNPL64V87T	RNRNN32X	2019-01-29	2019-02-24	26
420295	HUJS19JF55	RYSK6CTBSF68QHMPIOWC	RUNC4032	2019-02-23	2019-02-24	1
420296	HUJS19JF55	IFYS6Q0M7FSDI1LSDM44	RVC5VC50	2019-02-12	2019-02-24	12
420297	HUJS19JF55	JI0FUYT9E94PA5VT3PA0	RULLLL20	2019-02-02	2019-02-24	22
420298	HUJS19JF55	RN8IYP9SIL5MRXJ4EMEF	ROT2OT25	2019-02-10	2019-02-24	14
420299	HUJS19JF55	291HMO97S0M3KRORF50A	RTC5040X	2019-02-04	2019-02-24	20
420300	HUJS19JF55	07YGEL6JDRFRJEAENL9B	RUL4UL40	2019-02-04	2019-02-24	20

Figure 5.11. 648279 carrier data in database

The data was processed to create statistics and information that can be used to train the model.

id ↑	product_code	total_...	total_sh...	total_cost	total_price	total_wage	total_...	total_...	m...	average_demand	m...	m...	average_flow	m..	frequency
1	RB11B110	1496	22527	9574400	10621600	1047200	1083	340	15	4.4	1	5	1.38134810710988	1	0.931506849315069
2	RB12B125	1461	22094	11395800	12564600	1168800	1075	351	14	4.16239316239316	1	6	1.35906976744186	1	0.961643835616438
3	RB144016	1499	21693	11842100	13041300	1199200	1109	355	15	4.22253521126761	1	5	1.35166816952209	1	0.972602739726027
4	RB50RB50	1458	21510	5540400	6123600	583200	1070	346	13	4.21387283236994	1	5	1.36261682242991	1	0.947945205479452
5	RB63RB63	1535	22658	8289000	9210000	921000	1121	352	15	4.36079545454545	1	5	1.3693131132917	1	0.964383561643836
6	RB75RB75	1500	22663	14550000	16050000	1500000	1118	349	16	4.29799426934097	1	5	1.34168157423971	1	0.956164383561644
7	RB90RB90	1444	21518	3032400	3465600	433200	1087	347	16	4.16138328530259	1	6	1.32842686292548	1	0.950684931506849
8	RDB1B110	1489	22950	8636200	9529600	893400	1103	350	12	4.25428571428571	1	5	1.34995466908432	1	0.958904109589041
9	RDB2DB20	1449	21832	6955200	7679700	724500	1077	353	12	4.10481586402266	1	5	1.34540389972145	1	0.967123287671233
10	RDB2DB25	1503	22375	6462900	7214400	751500	1132	347	16	4.3314121037464	1	6	1.32773851590106	1	0.950684931506849
11	RDB3DB32	1420	21844	6958001	7668001	710000	1074	349	12	4.06876790830946	1	5	1.32216014897579	1	0.956164383561644

Figure 5.12. Statistic of each product synthetic from data

Then, the sum, maximum, mean, median, minimum, range, standard deviation, and variance values were calculated for each field and attribute.

id	type	carrier	age	cost	price	wage	day	maxd	avgd	mind	maxf	avgf	minf	frequency
1	sum	648279.0	9729529.0	34595590...	380808966...	34853059...	151648.0	6222.0	1855.379...	434.0	2229.0	588.7843...	434.0	415.473...
2	max	1626.0	24767.0	40908000...	449988000...	40908000...	362.0	21.0	4.619318...	1.0	8.0	1.417896...	1.0	0.99178...
3	avg	1493.730...	22418.26...	79713342....	87744001.5...	8030658.7...	349.4193...	14.33...	4.275068...	1.0	5.135...	1.356646...	1.0	0.95731...
4	med	1493.0	22419.0	8035250.0	8928350.0	893400.0	350.0	14.0	4.275071...	1.0	5.0	1.355492...	1.0	0.95890...
5	min	1380.0	20710.0	1501000.0	1651101.0	150101.0	338.0	11.0	3.923512...	1.0	4.0	1.306839...	1.0	0.92602...
6	ran	246.0	4057.0	40892990...	449822889...	40892989...	24.0	10.0	0.695805...	0.0	4.0	0.111057...	0.0	0.06575...
7	std	40.31641...	677.0471...	36043179...	396463094....	36031328....	3.914900...	1.684...	0.111816...	0.0	0.666...	0.019488...	0.0	0.01072...
8	var	1625.413...	458392.7...	12991107...	157182985...	12982566...	15.32644...	2.836...	0.012503...	0.0	0.444...	0.000379...	0.0	0.00011...

Figure 5.13. Sum, maxi, mean, median, min, range, std deviation, and variance

From the data gathered, two field of assign parameters and priority values are calculated.

assign	priority
16800.000000	94395.000000
218.573351	434.000000
77.419355	216.500000
78.000000	217.000000
11.000000	1.000000
207.573351	434.000000
10.389626	10.476163
107.944319	109.750000
0.478280	1.000000

Figure 5.14. Assign parameter and priority values

5.3.2 Parameter

Using the data for both input and output variables: sum, maximum, mean, median, minimum, range, standard deviation, and variance. Define the input antecedents, consequences and membership functions of each variables.

After many trials and errors, a sufficient number of rules (150) have been discovered. These rules fall between 1 and 729 (3^6) and satisfy the requirement of being neither too sparse to hinder decision making nor too robust to run.

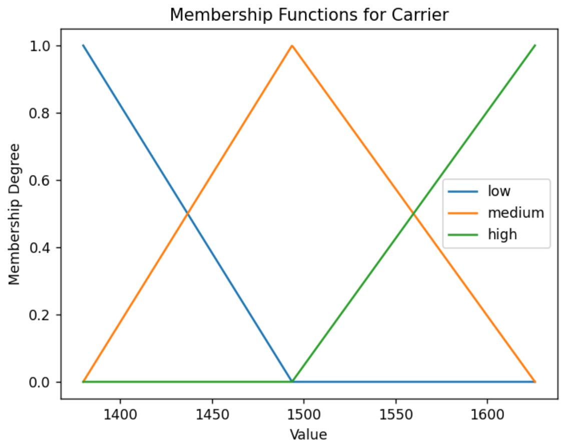


Figure 5.15. A graphical representation of input 1

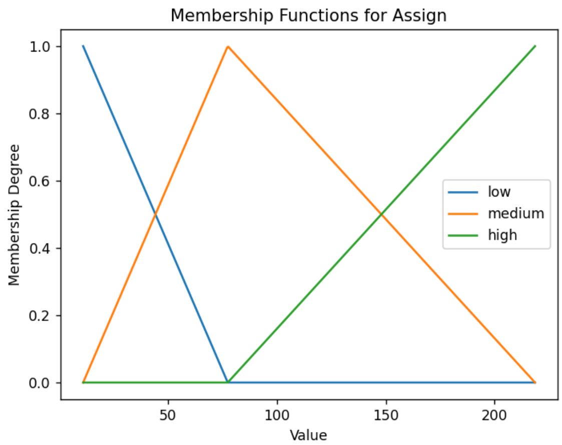


Figure 5.16. A graphical representation of output 1

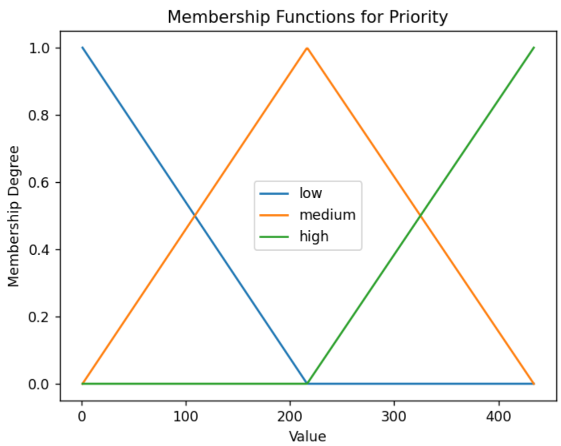


Figure 5.17. A graphical representation of output 2

After processing the rules, the control system will begin a simulation that takes inputs from generated data and returns output that includes the assigned parameters and priority values. After an optimization process, satisfactory results have been demonstrated.

```
Log file for generate_parameter.py
Start time: 2023-08-04 14:29:44.299399
150 rules read.
Control system.
Simulate system.
16795 slots allocated.
Priority check sum: 36053.53625463581.
Elsaped time: 59.81126689910889
```

Figure 5.18. The log result of the controller

The log results showed that with 150 rules in the control system, a total of 16,795 slots were successfully allocated as a multiple of 5 (the number of slots in a column), as required by the problem. Additionally, the priority checksum satisfied the condition of not exceeding the sum attribute of itself.

Since the data was generated by the machine, there is not much differentiation between each product, as shown by the standard deviation attributes of each field. Therefore, the space allocated for each product is not significantly different from one another.

```
Compute [409]: 410, RUTRRT25, 35, 75.4947795698434
Compute [410]: 411, RUTR51/2, 40, 82.18069209723804
Compute [411]: 412, RUV2UV20, 40, 90.07009948280827
Compute [412]: 413, RUV2UV25, 45, 98.90625139578498
Compute [413]: 414, RUV3UV32, 40, 91.06594504651605
Compute [414]: 415, RUV4UV40, 45, 103.52341337718362
Compute [415]: 416, RUV5UV50, 40, 85.83465024277957
Compute [416]: 417, RUVCVC20, 45, 101.94269830391268
Compute [417]: 418, RUVCVC25, 45, 103.30063960313146
Compute [418]: 419, RUVCVC32, 40, 93.33072116584873
Compute [419]: 420, RUVCVC40, 40, 88.14227782232858
Compute [420]: 421, RUVCVC50, 45, 102.3401204713904
Compute [421]: 422, RUVCVC63, 45, 97.94688141996005
Compute [422]: 423, RUZCZC20, 40, 80.19456367325989
Compute [423]: 424, RUZCZC25, 35, 73.8211768400468
Compute [424]: 425, RUZCZC32, 40, 80.59099038673843
Compute [425]: 426, RUZCZC40, 35, 77.79067123205851
Compute [426]: 427, RUZCZC50, 40, 80.59099038673843
Compute [427]: 428, RUZCZC63, 35, 75.82591117121113
Compute [428]: 429, RUZRRN20, 40, 81.36400372398134
Compute [429]: 430, RUZRRN25, 40, 78.91179291417578
Compute [430]: 431, RUZRRN32, 40, 80.33323953381881
Compute [431]: 432, RUZRRN40, 35, 74.85281188142994
Compute [432]: 433, RUZRRN63, 35, 74.18925172883371
Compute [433]: 434, RUZRRT25, 40, 82.40152966076433
```

Figure 5.19. The assign parameter computed for each products

CONCLUSION

In conclusion, this thesis aimed to investigate the structure and processes of Tien Phong's warehouse and create models to optimize and manage warehouse operations. Through this study, remedies have been proposed, and an independent warehouse management system has been devised for Tien Phong Plastic Joint Stock Company's warehouse. By implementing these proposed solutions, Tien Phong can improve its warehouse operations and enhance its overall efficiency and productivity.

However, further research, testing, development, and remediation are still required, as this project has many prospects to delve into. However, due to time constraints, these tasks could not be completed within the scope of this thesis.

REFERENCES

[1] J. Smith, *Warehouse Management: A Complete Guide to Improving Efficiency and Minimizing Costs in the Modern Warehouse*. Routledge, 2019.

[2] S. Abhijeet, K. Piyush, and P. Rahul, “Smart warehouse: A review on recent advancements,” *International Journal of Engineering Research and Technology*, vol. 8, no. 10, pp. 186–190, 2019.

[3] B. Duong, T. Le, and T. Nguyen, “Warehouse management system in plastic industry,” *International Journal of Engineering and Advanced Technology*, vol. 9, no. 1, pp. 1137–1141, 2019.

[4] V. Report, “Vietnam report: Vietnam plastic industry report 2021,” 2021.

[5] M. Jones, “The benefits of rack systems for smart warehouses,” *Supply Chain Management Review*, vol. 23, no. 1, pp. 54–55, 2019.

[6] T. P. Plastic. (2023) Tien phong plastic. Accessed on May 5, 2023. [Online]. Available: https://tienphongplastic.com.vn/

[7] S. Kumar and R. Singh, “A comprehensive review on warehouse management system,” *International Journal of Scientific Research in Computer Science, Engineering and Information Technology*, vol. 5, pp. 1–7, 2019.

[8] G. Richards, *Warehouse management: a complete guide to improving efficiency and minimizing costs in the modern warehouse*. Kogan Page Publishers, 2014.

[9] L. Wang, Y. Wang, and Y. Zhang, “A n-tier architecture for warehouse management system,” *International Journal of Computer Science Issues*, vol. 9, no. 2, pp. 125–132, 2012.

A: Tien Phong warehouse situation

A.1. Layout and design

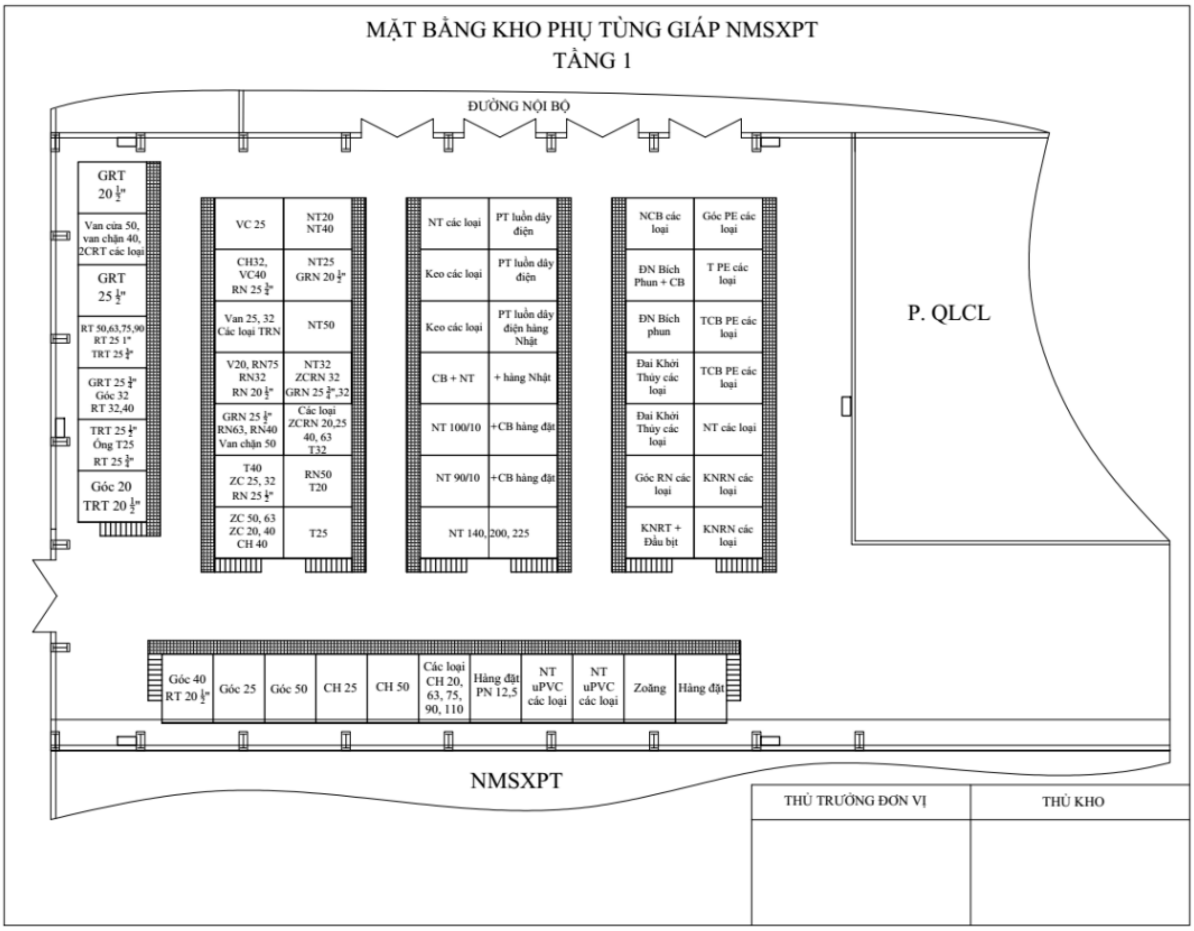


Figure A.1 Warehouse premises

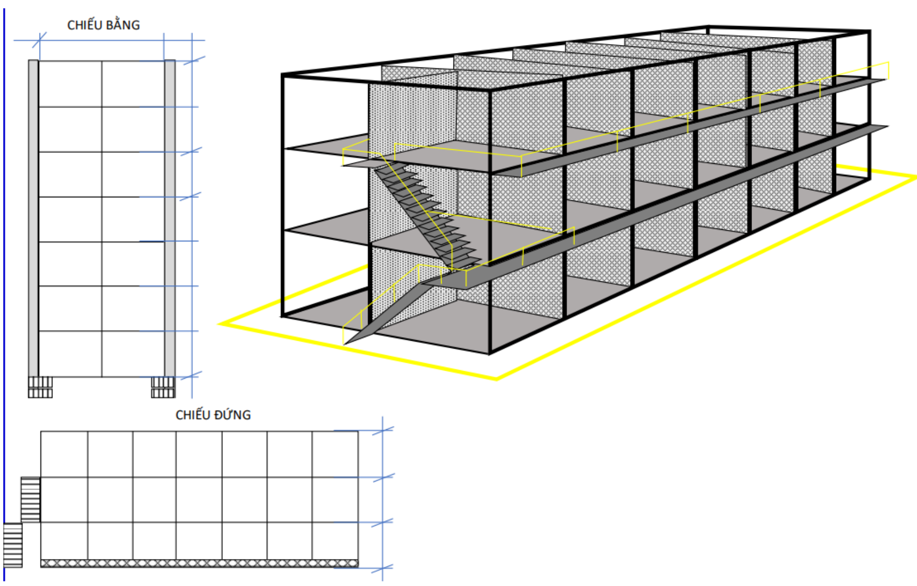


Figure A.2 Warehouse design

Table A.1. Storage capacity

Type	Location per floor	Number of floors	Number of locations	Capacity (m^3)
Single 7	7	3	21	504
Single 11	11	3	33	792
Double	14	3	42	1008

A.2. Management processes

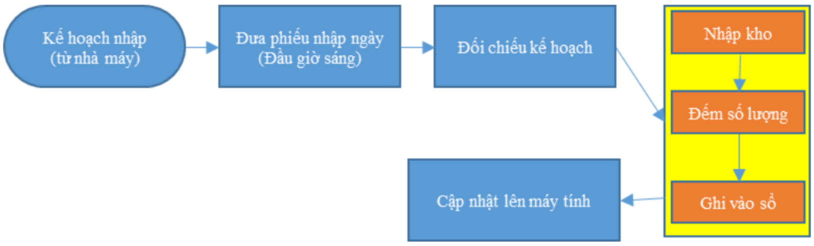


Figure A.3 Current receiving and put-away process

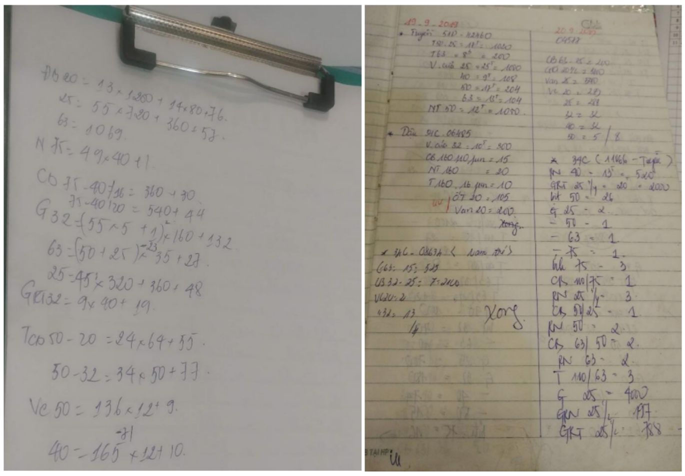


Figure A.4 Manual labour counting

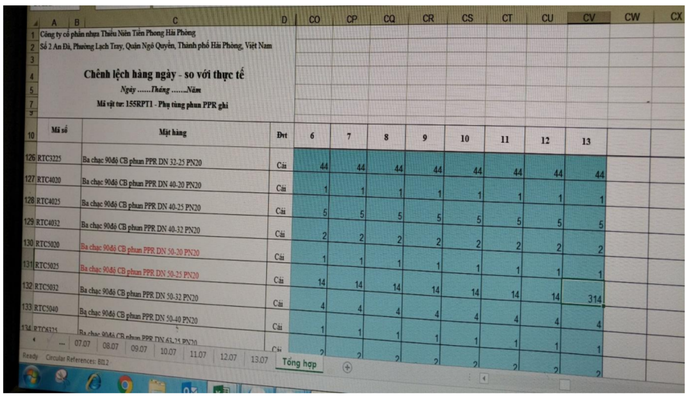


Figure A.5 Current way to record information

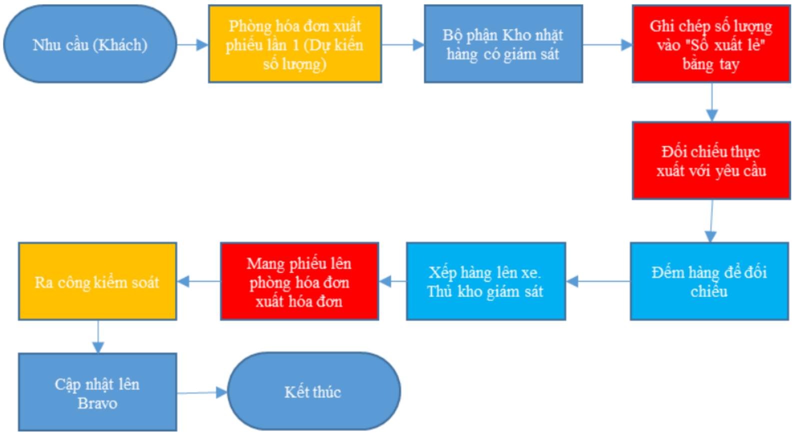


Figure A.6 Current order fulfillment process

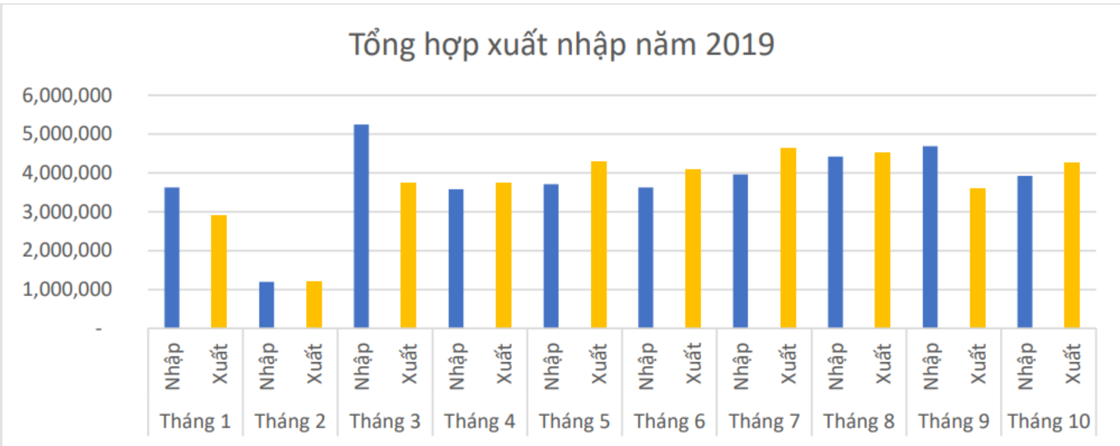


Figure A.7 Import and export report of 4 sections in 2019

Report shows that in the PP-R sections, the total amounts of imported and exported products (total products number is 434) are 37,991,108 and 36,982,276, respectively. The price range of PP-R products varies from a few thousand to several million VND for one product.

B: ABC analysis method

Prior to laying out a warehouse, deciding on the most appropriate handling equipment, installing storage systems and deciding on which form of picking system to introduce, a full ABC analysis of stock movements and stock held should take place.

Understanding ABC classification begins by understanding Pareto's Law or the 80/20 rule. Pareto's Law is widely used in logistics and is an excellent method for categorizing items. This is normally termed ABC classification.

It is states that roughly 80 percent of effects come from 20 percent of causes. This rule is not universal but it is surprising how often it can apply. The idea therefore is to concentrate time and resources on the important 20 percent or the 'vital few'.

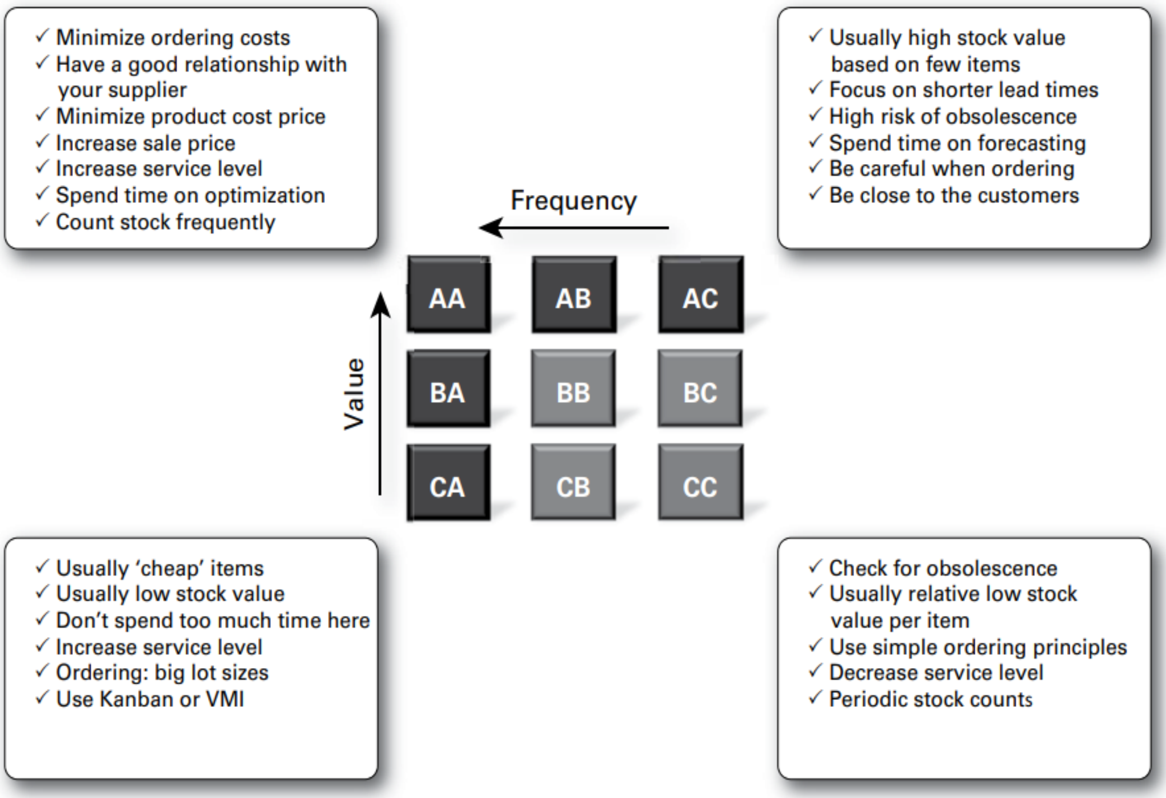


Figure B.1 ABC analysis: product value and frequency of sales